Building tidy R packages for the OMOP common data model

Edward Burn, Martí Català

2026-03-05

Table of contents

Preface **1**

I. Getting started **3**

1. Scope **5**
1.1. Is this package needed? 5

2. Ecosystem **6**
2.1. Motivation 6
2.2. Package groups 7
2.2.1. Core 7
2.2.2. Back-end 9
2.2.3. Analytics 9
2.2.4. Diagnostics 10
2.2.5. ETL 11
2.2.6. Visualisation 11
2.3. How the packages fit together 12
2.3.1. Imports 12
2.3.2. Imports and suggests 13
2.4. How your package fits in 14
2.5. Package statistics 15
2.6. Further reading 15

3. Contributing to a package **16**
3.1. GitHub issues 16
3.1.1. Opening GitHub issues 16

3.1.2. What makes a good issue . . . 16
3.2. Responding to GitHub issues . . . 17
3.3. Contributing code . . . 19
Contribute documentation . . . 19
Fixing a bug . . . 19
Adding new functionality . . . 19
3.4. Opening pull requests . . . 20
3.4.1. Initial checks . . . 21
3.4.2. Run tests . . . 21
3.4.3. Adhere to code style . . . 22
3.4.4. Run check() before opening a pull request . . . 22
3.4.5. Opening the pull request . . . 22
3.5. Reviewing pull requests . . . 23

4. Set up your environment **24**
4.1. Create an empty package . . . 25
4.1.1. The package structure . . . 25
4.2. Add a license . . . 26
4.3. Create a first function . . . 27
4.4. Check the package . . . 28
4.5. Create a simple test . . . 29
4.6. Create a README . . . 30
4.7. Create a documentation website . . . 31
4.8. Set up GitHub . . . 32
4.9. Using the package template . . . 32

II. Principles **33**

5. Core dependencies **36**
5.1. omopgenerics . . . 36
5.1.1. Classes . . . 36
5.1.2. Codelists . . . 38
5.1.3. The `summarised_result` class . . . 39

5.1.4. Methods . . . 39
5.1.5. Input validation functions . . . 40
5.1.6. Manipulating a `cdm_reference` . . . 41
5.1.7. Manipulating a `summarised_result` . . . 41
5.2. Other useful packages . . . 42
5.3. Imports vs Suggests . . . 42

6. Function interfaces **44**
6.1. Function naming . . . 44
6.1.1. Standard prefixes . . . 44
6.1.2. The `*CohortSet` suffix . . . 45
6.1.3. Internal functions . . . 46
6.2. Argument naming . . . 46
6.3. Argument order . . . 48
6.4. The `cdm` argument . . . 48
6.5. Boolean flags . . . 49
6.6. Dots (`...`) . . . 49

7. Conventions **50**
7.1. Casing . . . 50
7.2. Function naming . . . 51
7.2.1. Verb prefixes . . . 51
7.2.2. The `*CohortSet` suffix . . . 54
7.2.3. Noun component . . . 54
7.2.4. `mock*` functions . . . 54
7.3. Argument naming . . . 55
7.3.1. Standard argument names . . . 55
7.3.2. Argument order . . . 57
7.3.3. The `nameStyle` argument . . . 58
7.4. Windows . . . 59
7.5. Sex values . . . 60
7.6. Data values in `summarised_result` . . . 60
7.7. Column naming in cohort tables . . . 61
7.8. The `summarise` → `table`/`plot` pattern . . . 61

7.9. Inconsistencies to be aware of 62

8. Error messages and input validation 64
8.1. The two kinds of validation helpers in `omopgenerics` 64
8.2. `validate*` functions 65
8.2.1. Normalisation is the point 66
8.2.2. The `{{` syntax for `cohortId` 68
8.3. `assert*` functions 69
8.4. Putting it together: a validation block 70
8.5. Writing good error messages with `cli` 72
8.6. What to validate and what to skip 73
8.7. Summary 73

9. Function outputs 75
9.1. The `summarised_result` object 75
9.1.1. Structure 75
9.1.2. Settings attribute 77
9.1.3. How to create a `summarised_result` 78
9.1.4. Using the tidy format internally 79
9.1.5. Suppression 80
9.1.6. One `result_type` per function 80
9.1.7. Registering a `result_type` 80
9.2. Cohort outputs 81
9.3. Further reading 81

10. Visualisations 82
10.1. The `visOmopResults` API 82
10.2. Adding `table*` functions 83
10.3. Adding `plot*` functions 84
10.4. Checking what is already available 85
10.5. What not to do 85
10.6. Suggesting vs importing `visOmopResults` 85

11. Styling your code **86**
11.1. Code style 86
11.2. Linting 87
11.3. Formatting 88
11.4. Line length 88
11.5. Pipes 89
11.6. Assignment 89
11.7. Spacing 89

12. Reexporting from other packages **90**
12.1. When to re-export 90
12.2. When not to re-export 91
12.3. How to re-export 91
12.4. Implications of re-exporting 92
12.5. Checking your exports 92

III. Contributing documentation **93**

13. Documenting code **95**
13.1. When to document? 95

14. Documenting functions **96**
14.1. Title 96
14.2. Description 97
14.2.1. lifecycle badges 97
14.3. Parameters 97
14.4. Returns 98
14.5. Export 99
14.6. Examples 99

15. README **100**
15.1. Create the `.Rmd` file 100

15.2. Content . . . 101
15.2.1. Badges . . . 101
15.2.2. Tested sources . . . 102
15.2.3. Goal of your package . . . 103
15.2.4. How to install . . . 103

16. Vignettes **105**

17. Documentation website **106**

IV. Testing **107**

18. Introduction to testing **109**
18.1. The Basics . . . 109
18.1.1. Setting up test infrastructure . . . 109
18.1.2. Writing test blocks . . . 110
18.1.3. Running tests . . . 111
18.2. Expectations . . . 111
18.2.1. Common expectations . . . 111
18.2.2. Why expectations matter . . . 112
18.3. Setup and Helper Functions . . . 112
18.3.1. Helper files . . . 112
18.3.2. Setup and teardown . . . 113
18.3.3. Avoid top-level code . . . 113
18.4. What to Test . . . 114
18.4.1. General guidelines . . . 114
18.5. Documenting Tests . . . 114
18.5.1. Descriptive test names . . . 115
18.5.2. Commenting test logic . . . 115
18.5.3. Keeping tests readable . . . 115
18.6. Test Coverage . . . 116
18.7. Testing on CRAN . . . 117
18.8. Further reading . . . 118

19. Testing in the OMOP CDM **119**
19.1. Using omock to create mock synthetic data 119
19.1.1. Mock data 119
19.1.2. Mock datasets 119
19.1.3. Minimum viable dataset 119
19.2. Inserting the mock 120
19.3. Reusing mock data 120
19.4. Further reading 120

20. Testing against multiple DBMS **121**

V. GitHub **122**

21. Set up GitHub **124**

22. Automatic R-CDM checks **125**

23. Website deployment **126**

24. Test coverage **127**

25. Testing multiple DBMS **128**

VI. Maintaining a package **129**

26. Submit to CRAN **131**

Preface

This is an **opinionated** book on how we develop packages working in the Tidy R OMOP CDM space.

Our book, although covers some parts of basic R package development assumes the user knows the very basis. If you are new to R package development we would recommend you the following book: R Packages; Learn how to create a package, the fundamental unit of shareable, reusable, and reproducible R code.

The book is divided in four parts:

- **(I)** Getting started: covers how to start with an empty package, and helps you think about which is the scope of your package or guide you on how to contribute to existing packages.
- **(II)** Principles: this is the most important part of the book as it covers which packages to use, the different naming conventions, classes to use, how to validate the inputs and more.
- **(III)** Documentation: covers how to create the documentation of the packages, such as populate readme, documenting functions, documenting code, vignettes and more.
- **(IV)** Testing: covers how to write tests for the packages, what to test and how to do it efficiently.
- **(V)** GitHub: covers how to set up automatic GitHub testing and documentation using GitHub actions.

- **(VI)** Maintaining a package: once a package is ready for a first release you will have to maintain it, this packages covers how to submit to cran for first time and maintain the package for the future.

Part I.

Getting started

This first part of the book covers the decisions and steps that come before writing any package code. There are two distinct situations you might be in: starting a new package from scratch, or contributing to a package that already exists. Both are covered here.

The chapters are organised as follows:

- Scope asks the most important question of all: is this package actually needed? It walks through how to evaluate whether new functionality warrants a new package, whether it belongs inside an existing one, or whether it is already covered by the ecosystem.
- Ecosystem gives an overview of the full set of packages in the Tidy R OMOP CDM ecosystem. Understanding the landscape — which packages exist, what they do, and how they depend on each other — is essential before making scope decisions or choosing what to import.
- Contributing to a package covers the workflow for contributing to an existing package: how to open and respond to GitHub issues, how to write good pull requests, and what to expect from code review.
- Set up your environment walks through the practical steps to get a new package off the ground: creating the package skeleton, adding a licence, writing a first function, setting up tests, a README, a documentation website, and GitHub.

1. Scope

1.1. Is this package needed?

Before deciding to develop a package you must ask yourself many questions starting with: *Is this package needed?* A package must cover need and be refined. Are there other tools

2. Ecosystem

This chapter gives an overview of the ecosystem of R packages built for the OMOP Common Data Model (CDM) that your new package will become part of. Understanding the ecosystem — which packages exist, what they do, and how they relate to each other — is essential context before writing a single line of code. It will help you avoid reinventing existing functionality, identify the right packages to depend on, and make deliberate choices about where your package sits in the broader landscape.

2.1. Motivation

The OMOP CDM is a fixed, well-documented structure: every database mapped to it has the same tables, the same column names, and the same vocabulary conventions. This predictability is a major asset. It means that code written against one OMOP CDM instance should work against any other, regardless of the underlying database management system (DBMS) or institution.

The ecosystem described in this chapter exploits that predictability. Rather than writing raw SQL — which is verbose, fragile across different DBMS back-ends, and hard to compose — the packages in this ecosystem are built entirely in R, on top of dplyr and dbplyr. This means that:

- **No Java** and **no SQL** need to be written by package developers or end users.

- Analysis pipelines are expressed using familiar tidyverse-style code and the pipe operator.
- The same pipeline runs unchanged against DuckDB, PostgreSQL, SQL Server, Spark, and other back-ends.
- Packages are released on CRAN, making installation straightforward.

The foundation is the observation that dplyr verbs applied to a lazy `tbl` object (a remote table reference) are automatically translated to SQL by dbplyr and executed in the database. Package developers in this ecosystem work at the level of these lazy table references rather than materialising data into R unless absolutely necessary.

2.2. Package groups

The ecosystem is organised into six functional groups. Each group addresses a distinct concern, and packages within a group share conventions and interfaces.

2.2.1. Core

The core packages define the shared vocabulary — classes, methods, and synthetic data — that all other packages build on. Depending on one or more core packages is expected; depending on packages in other groups is a deliberate choice that should be made carefully (see Chapter 5).

Package	Purpose
`omopgenerics`	Defines the central classes (`cdm_reference`, `cohort_table`, `summarised_result`, codelists) and their generic methods. Every package in the ecosystem imports this.
`omock`	Creates synthetic OMOP CDM datasets for use in tests and examples. Pipeable API for building up only the tables you need.
`visOmopResults`	Provides tools for formatting and visualising `summarised_result` objects as tables and plots. Also used by analytics packages to present results.
`CodelistGenerator`	Generates and manages clinical code lists against the OMOP vocabulary. Also serves as the authoritative source for codelist-related operations across the ecosystem.
`PatientProfiles`	Adds patient-level feature columns (demographics, intersection flags, date-based variables) to any cohort table. Used internally by many analytics packages.

2.2.2. Back-end

Back-end packages handle database connectivity. They are responsible for creating the `cdm_reference` object — the central object of the whole ecosystem — against a particular database technology. End users typically load one back-end package; package developers should not need to depend on them directly.

Package	Purpose
`CDMConnector`	The primary back-end for connecting to any DBI-compatible database (DuckDB, PostgreSQL, SQL Server, Redshift, etc.).
`OmopOnDuckDB`	Specialised DuckDB back-end, including support for reading Parquet files directly.
`OmopOnPostgres`	Specialised PostgreSQL back-end, including support for native indexes.
`OmopOnSqlServer`	Specialised SQL Server and Synapse back-end.
`OmopOnSpark`	Back-end for Apache Spark deployments.

2.2.3. Analytics

Analytics packages implement the epidemiological methods that are the primary output of studies run on OMOP CDM data. They all take a `cdm_reference` and one or more `cohort_table` objects as input, and return results in the `summarised_result` format.

Package	Purpose
`CohortConstructor`	Builds and refines study cohorts from base clinical events.
`IncidencePrevalence`	Estimates incidence rates and prevalence from denominator and outcome cohorts.
`DrugUtilisation`	Characterises drug use: dose, duration, indication, discontinuation.
`CohortCharacteristics`	Summarises cohort demographics, comorbidities, and overlap between cohorts.
`CohortSurvival`	Estimates survival probabilities and competing risks from cohort data.
`CohortSymmetry`	Performs sequence symmetry analysis between two drug or exposure cohorts.
`OmopSketch`	Summarises the content of an OMOP CDM instance: record counts, trends, missing data. Also used as a diagnostic tool.

2.2.4. Diagnostics

Diagnostics packages assess the quality and fitness-for-purpose of codelists and cohorts before and after they are used in a study.

Package	Purpose
`CodelistGenerator`	Codelist diagnostics (also listed in Core).

Package	Purpose
`OmopSketch`	Database-level diagnostics (also listed in Analytics).
`PhenotypeR`	Runs a full suite of diagnostics at the database, codelist, cohort, and population level.
`MeasurementDiagnostics`	Diagnostics specific to codelists built from measurement and observation domains.

2.2.5. ETL

Package	Purpose
`OmopConstructor`	Builds derived OMOP CDM tables (observation period, drug era, etc.) from source data already in OMOP format. Also runs Achilles analyses.

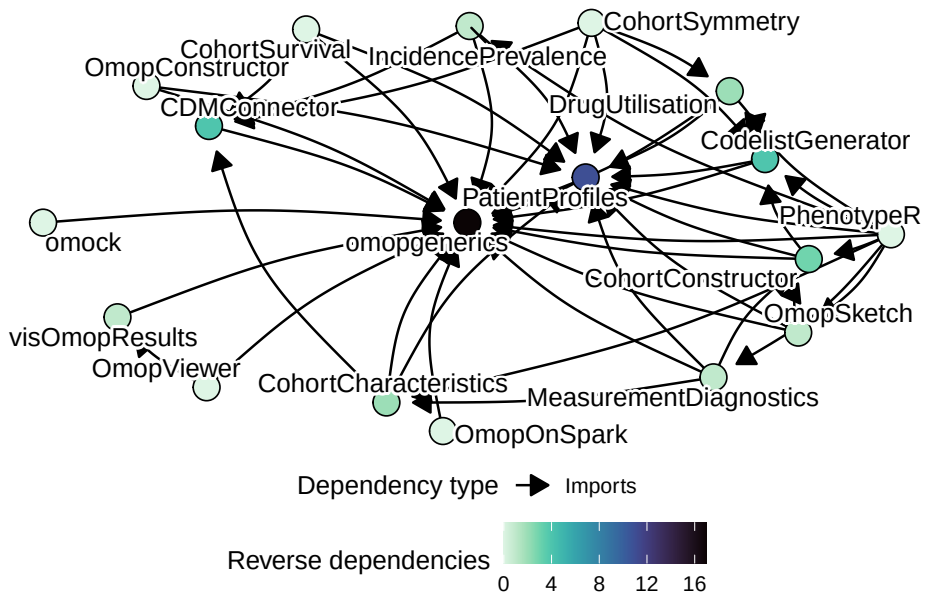
2.2.6. Visualisation

Package	Purpose
`visOmopResults`	Formatted tables and plots from `summarised_result` objects (also listed in Core).
`OmopViewer`	Generates interactive Shiny applications automatically from `summarised_result` objects.

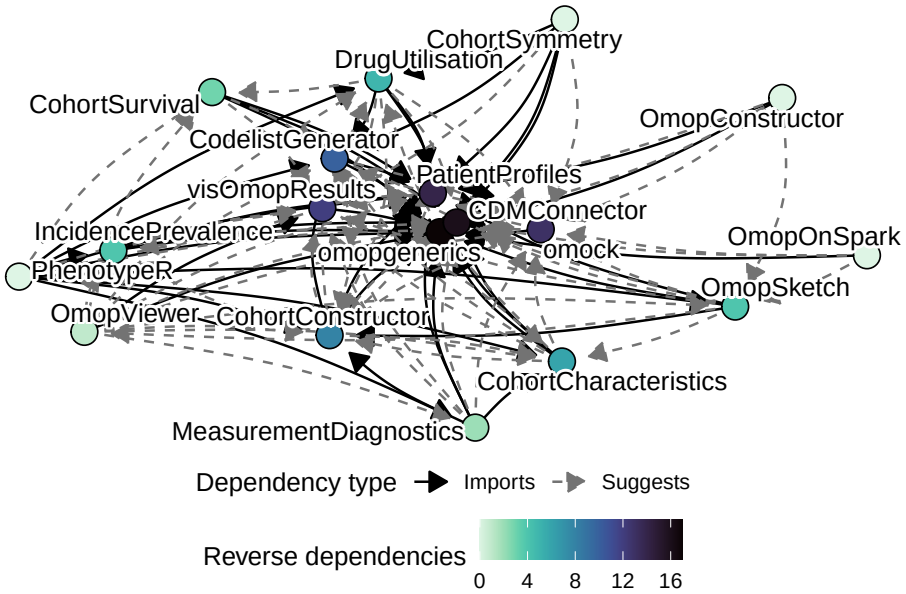
2.3. How the packages fit together

The diagram below captures the dependency structure of the ecosystem. `omopgenerics` is the central node: almost every package in the ecosystem imports it. `PatientProfiles` and `CodelistGenerator` sit just above it, providing functionality that is reused widely by analytics packages. The back-end packages and `omock` are largely independent of each other; they are connected to the rest of the ecosystem only through the shared `cdm_reference` class.

2.3.1. Imports



2.3.2. Imports and suggests



A key design principle is that the **result model is standardised**. All analytics packages return a `summarised_result` object (defined in `omopgenerics`). This means that `visOmopResults` and `OmopViewer` work without modification across every package that produces results — a developer working on, say, `CohortSurvival` does not need to write any bespoke visualisation code.

Similarly, the **cohort model is standardised**. A `cohort_table` produced by `CohortConstructor` can be passed directly to `IncidencePrevalence`, `DrugUtilisation`, `CohortCharacteristics`, or any other analytics package. The shared contract means pipelines can be composed without glue code.

2.4. How your package fits in

Before writing any code, locate your package in this map. There are a few common situations:

You are adding a new analytics method. Your package should import `omopgenerics` (for the `cdm_reference` and `cohort_table` classes), likely import `PatientProfiles` (for adding patient features to your cohort), and return results as a `summarised_result`. You should use `omock` and `CDMConnector` in your test suite but list them under `Suggests`, not `Imports`. See Chapter 5 for the distinction.

You are adding a new diagnostic tool. Similar to analytics, but you may also want to depend on `CodelistGenerator` for codelist management. Consider whether `PhenotypeR` already covers your use case, or whether your tool is genuinely complementary.

You are adding a new back-end. Your package should implement the `cdm_source` interface defined in `omopgenerics` (see `newCdmSource()`). Your only required import is `omopgenerics`; you should not depend on `CDMConnector` or other back-end packages.

You are adding a utility or helper package. If your package provides functionality that multiple other packages will use, it belongs in the Core group. Treat it as a potential dependency of many downstream packages and be especially conservative about what *it* depends on — every import you add is propagated to all packages that depend on you.

A practical rule of thumb: **the more packages that will depend on yours, the lighter your own dependency footprint should be.** A package that sits at the base of the dependency graph needs to be especially stable and minimal. A package at the periphery can afford to be more opinionated.

2.5. Package statistics

The ecosystem is under active development with frequent CRAN releases. As of 05 Mar 2026 there are around 22 packages in the ecosystem, 18 of which are on CRAN, with a combined total of 277 CRAN releases and 4.07984×10^5 total CRAN downloads.

The high release frequency is intentional. The ecosystem follows a continuous delivery philosophy: packages are kept in a releasable state at all times, which allows rapid iteration and fast response to bugs. As a new package developer you should expect to release early and often rather than waiting for a "complete" feature set.

2.6. Further reading

- The OxInfer software page provides an up-to-date summary of all packages, their CRAN versions, and a visualisation of the inter-package dependency graph.
- The ecosystem presentation gives a walkthrough of the history and structure of the ecosystem, including a detailed tour of `omopgenerics`.
- The Book of OHDSI provides essential background on the OMOP CDM itself.
- The companion book Tidy R programming with the OMOP Common Data Model covers how to *use* the ecosystem as a researcher, which is useful background even if your focus is package development.

3. Contributing to a package

3.1. GitHub issues

3.1.1. Opening GitHub issues

The first step to contributing to an existing package is by opening issues on its GitHub repository. These issues could be about bugs you encountered when using the package, requests for additional functionality (that you might even want to add yourself), or clarification questions on package documentation.

3.1.2. What makes a good issue

If reporting a bug, then a reprex makes the maintainers life much, much easier (and in turn increases the likelihood of a quick fix being introduced much higher!). This reprex will allow the developer to quickly reproduce the issue. This is already halfway to solving it, given the old adage > "A problem well stated is half solved."

Often an issue may arise when running code against a database with real patient-level data which can make it challenging to quickly create a reprex (as the patient data for which the bug was seen cannot be shared). In this situation it may be possible to reproduce the problem on an available synthetic dataset like Eunomia, in which case the package developer will be able to reproduce the bug with the synthetic data. However, it may be that you only encounter the issue for some particular set of patients

with specific characteristics which are not seen in the synthetic data. In this case you can, for example, use the omock package to create a set of synthetic patients with these characteristics so that the developer can use this same data to reproduce the problem.

Although of course it would be ideal to include a reprex, this may not always be possible. In such cases please do make sure to provide a bug report, with as much information as possible, as this will still be extremely useful and appreciated.

If opening an issue to ask for clarifications on documentation or with requests for additional functionality, the more precise and specific the issue the better. If you have a long laundry list of feature requests, it would generally be better to open each as separate issues so they can be addressed incrementally.

Note, even if you are not the maintainer of package but want to open a pull request to change documentation or add functionality then try to always open an issue to discuss this first. Even some seemingly trivial changes might be out of scope for the package (or they might have already been added in development branches of the repo). However well-intentioned, "drive-by pull requests" are often challenging for package maintainers to deal with.

3.2. Responding to GitHub issues

On the other side of the equation if you are the maintainer of package, receiving issues is one the main ways you will receive feedback on the package. Get used to starting with somewhat vague issues that will need some back and forth to get to the route of the problem or request for additional functionality.

When a bug is fixed in the package as a rule you should also add a corresponding test to make sure that it will stay fixed into the future. One

benefit of being given a reprex is that not only will it help you to fix the problem but the reprex itself can be used as the test for the package. By receiving and acting on issues raised, which will often be somewhat exotic edge cases or people using the package in some way that you hadn't expected, the package will become hardened over time.

When it comes to requests for adding functionality, or even offers to add it, this is a tricky balancing act. Remember that you will be the one maintaining the package in the long-term. It is often the case that the quick addition of something seemingly useful can make a package more difficult to maintain or extend in the future. Typically it is worth waiting a day or two at least before agreeing to add something to a package so that you don't rush into making an addition or change that you will later come to regret.

Another point to keep in mind when considering issues with requests for functionality is whether to add more general functionality than is being asked for. A well-intentioned user may be asking for something very specific to be added, but often underlying this request will be a broader need for more general functionality. This my tie in to other requests received, or may even need more discussion with users to better understand whether there is a broader need at play. general solutions

Remember that sadly few people be opening issues with praise if they've used your package successfully. The majority of users will use the code you've written without a problem and likely without even realising how much work has gone into it. And the people that do open issues are providing an extemely valuable service that will lead to the software getting better and better over time. So try to keep this in mind when tackling the seemingly never-ending list of issues you'll get if your software is being used.

3.3. Contributing code

Contribute documentation

Contributing documentation is one of the best ways to improve a package. Fixing typos, adding clarifications, and even writing whole vignettes will almost always be highly welcomed by the maintainer and improve the experience of other users. You can find issues related to documentation searching for the label *documentation*. Also you can open an issue if you think that somethings is not documented properly and propose the maintainer to fix it yourself.

You can read more details on how the documentation of the packages is done in the relevant chapters.

Fixing a bug

Contributing code is a more involved task. Before starting, make sure you've at the very least interacted with the maintainer so that they will be in favour of the changes or additions you are planning. You can do that replying to the issue you want to fix. Apart from fixing the bug, make sure to add tests alongside code changes to show that it is truly fixed. When fixing a bug existing tests should typically be unchanged (unless there is an issue with the test itself). If you are having to change existing tests to fix a bug this is normally a sign that there might be some more profound issue and will likely need back and forth between you and the maintainer for this to be addressed.

Adding new functionality

If adding functionality this again should be accompanied with tests to show that it adds the desired behaviour. Moreover, updates to package docu-

mentation, including changes or additions to vignettes might be needed. Expect back and forth with the package maintainer, as they might well have feedback on the implementation you propose.

3.4. Opening pull requests

Changes to packages should come via pull requests. Create a branch or a fork of the code, make your change, and open your pull request. If opening a pull request, more than anything try to create a pull request that will be easy for the package maintainer to review and merge. A pull request will be better for everyone involved when it addresses just one specific issue and affects less than 250 lines of code. One quick way to create complexity for the maintainer is to open a pull request that addresses multiple issues at the same time. Although this could be more efficient for the person opening the pull request, it often slows down the review by increasing the complexity of the review for the maintainer.

Before starting to contribute any code, first make sure the package tests are all passing. To do so after cloning run the following code:

```
devtools::check()
```

The output should be:

```
0 errors v | 0 warnings v | 0 notes v
```

If not raise an issue before going any further (although please first make sure you have all the packages from imports and suggests installed).

Now you are ready to do your code contribution. Add the relevant code and when you are happy with the changes that you have made, please follow these steps to open a pull request:

3.4.1. Initial checks

Run the below to update and check package documentation:

```
devtools::document()
devtools::check_man()
```

Test that the examples work:

```
devtools::run_examples()
```

Test that the readme and vignettes work and update the results if the output of any function has changed:

```
devtools::build_readme()
devtools::build_vignettes()
```

3.4.2. Run tests

As you then contribute code, make sure that all the current tests and any you add continue to pass. All package tests can be run together with:

```
devtools::test()
```

Code to add new functionality should be accompanied by tests. Code coverage can be checked using:

```
# note, you may first have to detach the package
# detach("package:IncidencePrevalence", unload=TRUE)
devtools::test_coverage()
```

3.4.3. Adhere to code style

Please adhere to the code style when adding any new code. Do not though restyle any code unrelated to your pull request as this will make code review more difficult.

```
lintr::lint_package(linters = lintr::linters_with_defaults(
  lintr::object_name_linter(styles = "camelCase")
))
```

3.4.4. Run check() before opening a pull request

Before opening any pull request please make sure to run:

```
devtools::check()
```

Please make sure that the output is the expected:

```
0 errors v | 0 warnings v | 0 notes v
```

Any error or warning will make your pull request actions fail. Although notes can pass the github action tests we encourage to fix all the notes as this is a requirement to keep the package on cran.

3.4.5. Opening the pull request

Once you've made sure that checks are passing and you are happy with your code additions you can open the pull request. When opening the pull request you must take into account the following:

- Write a meaningful title, titles of pull requests are later used to document the changes done. Make sure that the title of the pull request describes the issue that is fixed, therefore the documentation of the changes will be easier.
- Link the issue that your pull request is closing (we strongly encourage to fix issues one by one and do not include multiple issues in the same pull request). You can link a pull request and an issue using any of the gitub closing words or you can do it manually in the *development* section (bottom of the right sidebar).
- Describe any potential issue you want to remark. Help the maintainer review your pull request indicating if there is anything you are not sure about or that you want some feedback on. Don't hesitate to ask any question if needed.

3.5. Reviewing pull requests

Reviewing pull requests is a very important step that the maintainers have to do. When reviewing a pull request you have to find a good equilibrium between being kind (we want to encourage people to contribute to our packages) and being strict (you -as a maintainer- will be the ultimate responsible for that code; so review it carefully and only accept if you are happy about it and you feel comfortable maintaining that code).

4. Set up your environment

To build an R package you will need R installed and an IDE: preferably RStudio or Positron.

You will need to install devtools and usethis as they are key to follow the different steps to get started.

```r
# use base R
install.packages(c("usethis", "devtools"))

# or pak (recommended)
library(pak)
pkg_install(c("usethis", "devtools"))
```

To work with the Tidy R OMOP ecosystem, you will also need to install the omopgenerics package. Alongside with other packages that may be useful:

- PatientProfiles for patient-level feature extraction.
- CodelistGenerator to query the OMOP vocabularies.
- visOmopResults for formatting and visualising results.

You can see the full list of Tidy R packages in our website.

4.1. Create an empty package

To create a new empty package, use `usethis::create_package()` with the path to the folder where you want the package to live. The folder name becomes the package name.

```r
library(usethis)

create_package(path = "path/to/MyPackage")
```

This will:

- Create the folder `MyPackage/` with the standard package structure (`R/`, `DESCRIPTION`, `NAMESPACE`).
- Open the new project in a fresh RStudio or Positron window.

> **Choosing a package name**
>
> Package names must start with a letter, contain only letters, numbers, and dots, and ideally be short, distinctive, and easy to remember. Use CamelCase to make multi-word names readable (e.g. `CohortConstructor`, `PatientProfiles`). Avoid dots, underscores, and all-lowercase names that are hard to distinguish.
> You can check whether a name is available on CRAN with `available::available("MyPackageName")`.

4.1.1. The package structure

After `create_package()` you will have:

```
MyPackage/
  R/                    # Your R source files go here
```

```
DESCRIPTION         # Package metadata (name, version, dependencies, …)
NAMESPACE           # Exported functions - managed automatically by devtools
MyPackage.Rproj     # RStudio project file
```

You will add more files as you go: `tests/`, `vignettes/`, `man/`, and `NEWS.md` among others. Most of these are created for you by `usethis` helpers described later in this chapter.

4.2. Add a license

Adding a license is essential — it tells users how they can use, modify, and redistribute your package, and protects you legally. In the Tidy R OMOP ecosystem we release under the Apache 2.0 license.

```
use_apache_license()
```

This adds the `LICENSE.md` file and populates the `License` field in `DESCRIPTION`.

> **License options**
>
> `usethis` supports all common open-source licenses. The right choice usually comes down to how permissive you want to be:
>
> - More permissive (anyone can use your code, even in closed-source projects):
> - MIT: simple and widely understood.
> - Apache 2.0: like MIT but adds explicit patent protection. This is our default.
> - Copyleft (anyone who distributes changes must also open-source them):

  - GPL v3
  - AGPL v3: extends GPL to cover use over a network.
  - LGPL v3: a weaker copyleft, suitable for libraries.

- Creative Commons (appropriate for data packages rather than code):
  - CC0: dedicated to the public domain.
  - CC-BY: free to share and adapt, with attribution.

```
use_mit_license()
use_apache_license()
use_gpl_license(version = 3)
use_agpl_license(version = 3)
use_lgpl_license(version = 3)
use_cc0_license()
use_ccby_license()
```

4.3. Create a first function

All R source files live in the `R/` folder. Each file can contain one or more functions — the typical convention is one file per function or per closely related group of functions, named to match.

Create a new R file with:

```
use_r("myFunction")
```

This creates `R/myFunction.R` and opens it. Write your function there:

```r
myFunction <- function(cohort, overlap = FALSE) {
  # your code here
  cohort
}
```

> **Add a roxygen skeleton immediately**
>
> Before writing the body, add a roxygen2 documentation skeleton with **Code → Insert Roxygen Skeleton** in RStudio (or `Ctrl+Alt+Shift+R`). Fill in the `@title`, `@description`, `@param`, and `@return` tags as you write the function. Doing this from the start is much easier than adding it later. Documentation is covered fully in **?@sec-functions**.

Once you have written a function, load the whole package into your session with:

```r
devtools::load_all()  # Ctrl+Shift+L in RStudio
```

`load_all()` simulates what happens when the package is installed and loaded — it makes all your functions available without having to restart R. Use it constantly during development.

4.4. Check the package

`devtools::check()` runs `R CMD check`, which is the official test that CRAN and you should run to verify your package is correct. It checks documentation, dependencies, examples, and tests.

```r
devtools::check()  # Ctrl+Shift+E in RStudio
```

A clean check produces output ending with:

```
  R CMD check results
Duration: 30s

0 errors   | 0 warnings   | 0 notes
```

Aim to keep your package at **0 errors, 0 warnings, 0 notes** at all times — not just before a release. It is much easier to fix one new note immediately than to diagnose five notes that accumulated over months.

Common things to fix early:

- A missing `@export` tag on a function that should be public.
- An undocumented argument (`@param` entry missing).
- A package used in code but not listed in `DESCRIPTION` under `Imports` or `Suggests`.
- An example that fails because it requires a real database.

4.5. Create a simple test

Tests live in `tests/testthat/`. Set up the testing infrastructure with:

```r
use_testthat()
```

Then create a test file for your function:

```r
use_test("myFunction")
```

This creates `tests/testthat/test-myFunction.R`. A minimal test looks like:

```
test_that("myFunction returns a cohort table", {
  cdm <- mockPatientProfiles()
  result <- myFunction(cdm$cohort1)
  expect_s3_class(result, "cohort_table")
  CDMConnector::cdmDisconnect(cdm)
})
```

Run the tests for the file you are currently editing:

```
devtools::test_active_file()  # Ctrl+Shift+T in RStudio (for active file)
```

Or run all tests in the package:

```
devtools::test()
```

Testing with OMOP data is covered in detail in **?@sec-test-omop**. For now, the important habit is: every function you write should have at least one test file created for it before you move on.

4.6. Create a README

The README is the first thing people see when they visit your package on GitHub. Create one with:

```
use_readme_rmd()
```

This creates `README.Rmd`, which is an R Markdown file that can contain executable code chunks. After editing it, render it to produce `README.md`:

```
devtools::build_readme()
```

Always commit both `README.Rmd` and `README.md`. GitHub displays `README.md` automatically on the repository front page.

A good README for an ecosystem package should include: a one-paragraph description, an installation code block, a minimal usage example using a `mock*` function (no real database needed), and links to the documentation website. The full README conventions are covered in **?@sec-readme**.

4.7. Create a documentation website

A pkgdown website gives your package a searchable, browsable reference site generated automatically from your roxygen2 documentation and vignettes. Set it up with:

```
use_pkgdown()
```

Build the site locally to preview it:

```
pkgdown::build_site()
```

This creates a `docs/` folder. Open `docs/index.html` in your browser to see it.

The ecosystem uses a shared pkgdown theme for visual consistency. Configuration and automated deployment via GitHub Actions are covered in **?@sec-automatic-website**.

4.8. Set up GitHub

Once your package is working locally, push it to GitHub so it can benefit from continuous integration: automated checks, test coverage reporting, and website deployment on every push.

The fifth part of this book covers GitHub and GitHub Actions in full:

- Automatic checking your package
- Automatic deployment of a documentation website
- Automatic assessment of test coverage
- Testing across multiple database backends

4.9. Using the package template

Although it is instructive to follow these steps manually the first time, the ecosystem provides a GitHub template repository that does all of this setup for you. Use it to create a new repository with the full scaffold already in place — license, GitHub Actions, pkgdown configuration, and lintr — by clicking the link below:

Create a new package from the template

> **Template repository**
>
> The source for the template is at https://github.com/oxford-pharmacoepi/EmptyPackageTemplate. You can inspect it to see exactly what files are included.

Part II.

Principles

This part of the book is the most important. It sets out the conventions and design principles that all packages in the Tidy R OMOP CDM ecosystem follow. Adhering to these principles is what makes the ecosystem feel coherent: a developer who has worked with one package will find the next one immediately familiar, and an end user can compose functions from different packages into a single pipeline without friction.

These principles are not arbitrary. Each one exists because of a concrete problem encountered as the ecosystem grew: inconsistent naming caused confusion, ad-hoc result formats made downstream visualisation impossible, duplicated validation logic introduced divergent behaviour. The goal of this part is to give you the reasoning behind each convention, not just the rule.

The chapters are organised as follows:

- Core dependencies describes the packages that sit at the foundation of the ecosystem — above all `omopgenerics` — and explains the classes, methods, and utilities they provide. Knowing what is already available prevents you from reimplementing it.
- Function interfaces covers the naming conventions for functions and arguments, the standard function prefixes used across the ecosystem, and the design principles that make functions easy to compose.
- Conventions describes lower-level code style conventions: casing, argument naming, and the rules that apply consistently across all packages.
- Error messages and input validation explains how to validate function inputs in a consistent and user-friendly way, making use of the validation helpers provided by `omopgenerics`.
- Function outputs introduces the `summarised_result` object — the standard result format returned by all analytics functions in the ecosystem — and explains how to create, populate, and register result types correctly.

- Visualisations covers how to produce tables and plots from `summarised_result` objects using `visOmopResults`, and the conventions packages should follow when adding `table*` and `plot*` functions.
- Styling your code describes the code formatting standards used across the ecosystem and the tools used to enforce them.
- Reexporting from other packages explains when and how to reexport functions from upstream packages, and the implications of doing so.

5. Core dependencies

The *Tidy R OMOP CDM* packages rely on dplyr and dbplyr to manipulate data from a `cdm_reference` object. The `cdm_reference` is the central object in the ecosystem — a named list of lazy database table references, all pointing to the same underlying data source. It is defined in omopgenerics, the package that every other package in the ecosystem depends on.

This chapter describes what `omopgenerics` provides and which other core packages are useful enough to consider importing or suggesting.

5.1. omopgenerics

`omopgenerics` defines the classes and methods that give the ecosystem its coherence. It is a deliberately minimal package: its only external dependencies are `dplyr`, `dbplyr`, `cli`, `glue`, and `rlang`. This keeps its footprint small so that depending on it does not burden downstream packages.

5.1.1. Classes

The four main classes defined by `omopgenerics` are:

`cdm_reference` is the central object of the whole ecosystem. It is a named list whose elements are lazy table references pointing to OMOP CDM tables in a database, along with any cohort or other tables associated with the CDM. The `cdm_reference` knows which database back-end it is connected to, what the CDM name is, and what schema to write results

into. Every analytical function in the ecosystem takes a `cdm_reference` (or a table derived from one) as its first argument.

```
library(omopgenerics)

# Access CDM properties
cdmName(cdm)
cdmVersion(cdm)
cdmSource(cdm)
sourceType(cdm)

# Insert, read, list, and drop tables in the write schema
cdm <- insertTable(cdm, name = "my_table", table = my_data)
cdm <- readSourceTable(cdm, name = "my_table")
listSourceTables(cdm)
cdm <- dropSourceTable(cdm, name = "my_table")
```

`cdm_table` is the class of any table that belongs to a `cdm_reference`. All CDM tables carry a back-reference to their parent `cdm_reference`, which means you can always navigate from a table back to the full CDM:

```
cdmReference(cdm$person)   # returns the full cdm_reference
cdmName(cdm$person)        # works just like cdmName(cdm)
tableName(cdm$person)      # returns "person"
```

`omop_table` is a subclass of `cdm_table` that protects the integrity of the standard OMOP CDM tables. It prevents removal of required columns and enforces that a table assigned to `cdm$person` must actually be named `person`. Filtering, mutating, and adding columns are all permitted — only operations that would break the CDM schema are blocked.

```r
# This is blocked - person_id is required
cdm$person <- cdm$person |> select(-person_id)
#> Error: person_id is not present in table person

# This is fine - adding a column is allowed
cdm$person <- cdm$person |> mutate(extra_col = "my_content")
```

`cohort_table` extends `cdm_table` with the structure required for a study cohort. A cohort table has exactly four columns (`cohort_definition_id`, `subject_id`, `cohort_start_date`, `cohort_end_date`) and carries three mandatory attributes stored as database tables: the cohort set (metadata per cohort), the attrition log (a record of how many records and subjects were retained at each step), and the cohort codelist (the clinical codes used to define each cohort).

```r
settings(cdm$my_cohort)      # cohort set: name, cdm_version, vocabulary_version
cohortCount(cdm$my_cohort)   # records and subjects per cohort
attrition(cdm$my_cohort)     # full attrition table

# Record a new attrition step after filtering
cdm$my_cohort <- cdm$my_cohort |>
  filter(subject_id %% 2 == 0) |>
  compute(name = "my_cohort") |>
  recordCohortAttrition("Only even subjects")
```

5.1.2. Codelists

`omopgenerics` defines three classes for representing clinical code lists:

- **`codelist`**: a named list of integer vectors, one per concept set.
- **`codelist_with_details`**: a named list of tibbles, where each tibble contains `concept_id` alongside additional properties (e.g. `concept_name`, `domain_id`).

- **concept_set_expression**: a named list of tibbles with concept_id, excluded, descendants, and mapped columns — matching the ATLAS JSON format and the OHDSI TAB guidelines.

```
# Create a simple codelist
cl <- newCodelist(list(hypertension = c(316866L, 320128L), diabetes = c(201826L)))

# Combine and subset
c(cl, newCodelist(list(asthma = 317009L)))
cl["hypertension"]

# Export and import (JSON is compatible with ATLAS)
exportCodelist(cl, path = here::here(), type = "json")
cl <- importCodelist(path = here::here(), type = "json")
```

In general, analytical functions in the ecosystem accept a conceptSet argument that can be any of these three classes (or a plain named list of integers that will be coerced to a codelist).

5.1.3. The summarised_result class

summarised_result is the standard result format for all analytics functions in the ecosystem. It is a tibble with 13 compulsory columns. See Chapter 9 for a full description and guidance on how to create and populate it.

5.1.4. Methods

omopgenerics exports a set of generic S3 methods that packages implement for their own classes:

Generic	Purpose
`cdmName()`	Return the name of the CDM instance
`cdmVersion()`	Return the CDM version
`cdmSource()`	Return the `cdm_source` object
`sourceType()`	Return the back-end type (e.g. `"duckdb"`)
`tableName()`	Return the name of a `cdm_table`
`settings()`	Return cohort set metadata
`cohortCount()`	Return record/subject counts per cohort
`attrition()`	Return the attrition log
`cohortCodelist()`	Return the codelist used to build a cohort
`recordCohortAttrition()`	Append a step to the attrition log
`bind()`	Bind two `summarised_result` objects together
`suppress()`	Apply small-number suppression to a `summarised_result`

5.1.5. Input validation functions

`omopgenerics` provides `validate*` functions for the arguments that appear repeatedly across the ecosystem. Using these instead of writing bespoke validation code keeps error messages consistent across packages. See Chapter 8 for full details and examples. The available validators are:

`validateCdmArgument()`, `validateCohortArgument()`, `validateCohortIdArgument()`, `validateConceptSetArgument()`, `validateNameArgument()`, `validateNameStyle()`, `validateResultArgument()`, `validateStrataArgument()`, `validateWindowArgument()`, `validateAgeGroupArgument()`.

There are also `assert*` helpers for simpler type checks: `assertCharacter()`, `assertNumeric()`, `assertLogical()`, `assertDate()`, `assertList()`, `assertClass()`.

5.1.6. Manipulating a `cdm_reference`

In addition to the methods above, `omopgenerics` provides utilities for working with CDM table collections:

```
# List tables by type
omopTables(cdm)       # standard OMOP CDM tables
cohortTables(cdm)     # cohort tables in the write schema
achillesTables(cdm)   # Achilles result tables

# Insert and drop
cdm <- insertTable(cdm, name = "temp", table = my_tibble)
cdm <- dropSourceTable(cdm, name = "temp")
```

5.1.7. Manipulating a `summarised_result`

A `summarised_result` object has its own set of helper functions:

```
# Inspect
settings(result)          # result-level metadata (result_type, package_name, ...)
groupColumns(result)      # columns encoded in group_name / group_level
strataColumns(result)     # columns encoded in strata_name / strata_level

# Transform
tidy(result)              # pivot to wide format - useful for internal computation
bind(result1, result2)    # combine two summarised_result objects
suppress(result, minCellCount = 5)  # apply cell suppression
```

5.2. Other useful packages

Beyond `omopgenerics`, several other core packages are commonly useful. Unlike `omopgenerics`, these should generally be listed under `Suggests` unless your package's primary purpose depends on them.

PatientProfiles adds columns to cohort tables: demographics (age, sex, prior observation, future observation), intersection flags and values (e.g. "was the patient on drug X at cohort start?"), and date-relative summaries. Browse the function reference before implementing any feature-derivation logic — it is very likely already there.

visOmopResults transforms `summarised_result` objects into formatted tables and `ggplot2` plots. The package supports multiple output formats for tables (gt, flextable, tibble) with a consistent API. See Chapter 10 for guidance on how to use it in your package. Browse the function reference before adding any result-formatting code.

CodelistGenerator generates codelists by searching the OMOP vocabulary, and provides utilities for subsetting, stratifying, comparing, and diagnosing codelists. If your package works with clinical concepts, check the function reference first.

omock creates synthetic OMOP CDM datasets for testing. It should almost always be listed under `Suggests`, not `Imports`. See **?@sec-test-omop** for detailed guidance on using it in your test suite.

5.3. Imports vs Suggests

A package listed under `Imports` is loaded automatically when your package is loaded. A package listed under `Suggests` is not — it must be available but is only used in specific circumstances (tests, vignettes, or optional code paths guarded by `requireNamespace()`).

5. Core dependencies

The key rule for this ecosystem:

- **Always `Imports`**: `omopgenerics`, any package whose classes or generics you use in exported functions.
- **Usually `Suggests`**: `omock` (tests only), `CDMConnector` / back-end packages (tests only), `visOmopResults` (unless your package's core output is visualisation), `PatientProfiles` (unless your package is built on top of it).

When in doubt, prefer `Suggests`. A lighter `Imports` list means fewer forced transitive dependencies for your users.

6. Function interfaces

Consistent function interfaces are one of the most visible ways the ecosystem signals coherence to developers and end users alike. When every package follows the same naming conventions, a user who has worked with one package can immediately form correct expectations about another. This chapter sets out the conventions used across the ecosystem.

6.1. Function naming

Functions in the ecosystem follow a **verb-noun** pattern using **camelCase**. The verb indicates what the function does; the noun indicates what it operates on or returns.

6.1.1. Standard prefixes

The ecosystem uses a small set of standard verb prefixes. Using the right prefix for a given function type is important because it communicates intent and enables consistent user mental models.

Prefix	Purpose	Examples
`generate*`	Create a new cohort table in the CDM write schema	`generateConceptCohortSet()`, `generateDemographicCohortSet()`

Prefix	Purpose	Examples
`add*`	Add columns to an existing table (returns the table with extra columns)	`addAge()`, `addSex()`, `addIntersect()`
`summarise*`	Compute aggregated results; returns a `summarised_result`	`summariseCharacteristics()`, `summariseDrugUtilisation()`
`plot*`	Create a `ggplot2` visualisation from a `summarised_result`	`plotIncidence()`, `plotSurvival()`
`table*`	Create a formatted table from a `summarised_result`	`tableCharacteristics()`, `tableDrugUtilisation()`
`compute*`	Materialise a lazy query into a temporary database table	`computeQuery()`

> **Note**
>
> The `generate*` prefix is reserved for functions that write a new cohort table to the database. Functions that merely manipulate an existing cohort in memory (filtering, unioning, etc.) without necessarily writing to the database should not use this prefix.

6.1.2. The `*CohortSet` suffix

Functions that create a full `cohort_table` with potentially multiple cohorts use a `*CohortSet` suffix:

```
generateConceptCohortSet(cdm, conceptSet = my_codelist, name = "my_cohort")
generateDemographicCohortSet(cdm, ageGroup = list(c(0, 17), c(18, 64)), name = "age_
```

Functions that operate on an existing cohort table and return a modified version of it (e.g. applying an inclusion criterion) do not use the `Set` suffix:

```
requireIsFirstEntry(cohort)
requireAge(cohort, ageRange = c(18, Inf))
```

6.1.3. Internal functions

Internal (unexported) functions should still follow camelCase but are conventionally prefixed with a dot or given a descriptive name that makes clear they are not part of the public API. Functions documented with `@noRd` are not exported and do not appear in the package documentation website.

6.2. Argument naming

Arguments should also use camelCase. The following argument names are standardised across the ecosystem and should be used whenever they apply:

Argument	Type	Description
`cdm`	`cdm_reference`	The CDM reference object. Always the first argument.
`cohort`	`cohort_table`	A cohort table.

Argument	Type	Description
`cohortId`	`integer` or `NULL`	IDs of cohorts to operate on; `NULL` means all cohorts.
`conceptSet`	`codelist` / `codelist_with_details` / `concept_set_expression`	A set of clinical concepts.
`name`	`character(1)`	Name for the output table to be written to the CDM.
`nameStyle`	`character(1)`	A glue-style string for naming multiple output columns.
`strata`	`list` of `character` vectors	Stratification variables.
`ageGroup`	named list or `NULL`	Age group definitions.
`window`	`list` of integer vectors	Time windows relative to an index date.
`overlap`	`logical(1)`	Whether overlapping records should be merged.
`minCellCount`	`integer(1)`	Minimum cell count for suppression. Default `5`.

When adding new arguments, check first whether a standard name already exists in `omopgenerics` or in widely used packages — reusing names keeps the interface predictable.

6.3. Argument order

Arguments should be ordered as follows:

1. `cdm` (if present) — always first.
2. `cohort` or other primary data argument.
3. `cohortId` (if present) — immediately after its parent `cohort`.
4. Content arguments that define *what* to compute.
5. Arguments that modify *how* to compute (stratifications, windows, age groups).
6. `name` — the name of the output table, near the end.
7. `...` — rarely needed; avoid unless implementing a generic.

6.4. The `cdm` argument

Almost every exported function in an analytics or diagnostics package takes `cdm` as its first argument. This makes functions pipe-friendly in the sense that the CDM is always at the root of an analysis, and it makes the interface immediately recognisable.

```
# Standard pattern
result <- summariseCohortOverlap(
  cohort = cdm$my_cohort,
  cohortId = NULL,
  strata = list(c("sex"), c("age_group")),
  minCellCount = 5
)
```

Note that for functions operating on a `cohort_table`, the `cdm_reference` is accessible through `cdmReference(cohort)`, so it is not always necessary to accept `cdm` as a separate argument. Functions whose primary input is a cohort table can accept just `cohort`:

```
# cohort-first pattern - cdm is accessible via cdmReference(cohort)
requireAge(cohort, ageRange = c(18, Inf))
```

6.5. Boolean flags

Boolean arguments should default to `FALSE` unless `TRUE` is the overwhelmingly common case. Argument names should be positive statements (prefer `overlap = TRUE` over `noOverlap = FALSE`). Avoid arguments that accept a string where a boolean would do.

6.6. Dots (...)

Avoid `...` in the interfaces of exported functions unless you are implementing an S3 generic. Dots make it easy to silently swallow misspelled argument names, which leads to confusing behaviour.

7. Conventions

This chapter documents the naming and structural conventions observed across the ecosystem. Many will feel familiar if you have already worked with the packages, but having them written down in one place makes it easier to apply them consistently in new code and to identify deviations worth addressing.

Where real inconsistencies exist across the current packages, they are noted — the goal is for new packages to conform to these conventions even when older ones pre-date them.

7.1. Casing

The ecosystem uses **camelCase** for all R identifiers: exported function names, unexported helper function names, argument names, and local variable names.

```
# Correct - camelCase throughout
generateIngredientCohortSet(cdm, name = "dus_cohort", gapEra = 7)
addCohortIntersectFlag(cohort, targetCohortTable = "cohort2", window = c(-Inf, -1))
summariseClinicalRecords(cdm, omopTableName = "condition_occurrence")
```

The camelCase rule is enforced by lintr:

```r
lintr::lint_package(
  linters = lintr::linters_with_defaults(
    lintr::object_name_linter(styles = "camelCase")
  )
)
```

Exception: strings stored in data. Column names in database tables, `variable_name` values inside `summarised_result` objects, `result_type` strings, cohort names, and table names in the CDM are all `snake_case`. The camelCase rule applies to R object names and arguments, not to string values that end up in data or in the CDM.

```r
# The R argument is camelCase...
addAge(cohort, ageGroup = list(c(0, 17), c(18, 64)))

# ...but values stored as data are snake_case:
# group_level  = "acetaminophen"
# variable_name = "number_subjects"
# result_type   = "summarised_characteristics"
# cohort name   = "dus_cohort"
```

7.2. Function naming

7.2.1. Verb prefixes

Every exported function begins with a standard verb prefix. The prefix is not merely a naming choice — it signals the return type, the side effects, and the role the function plays in a pipeline.

Prefix	Returns	Side effects	Representative examples
`generate*`	`cohort_table` written to CDM	Writes table to database	`generateConceptCohortSet()`, `generateIngredientCohortSet()`, `generateDenominatorCohortSet()`
`require*`	Modified `cohort_table`	May rewrite cohort table in DB; updates attrition	`requireIsFirstEntry()`, `requireAge()`, `requireSex()`, `requireCohortIntersect()`, `requireConceptIntersect()`
`add*`	Input table with extra columns appended	None	`addAge()`, `addSex()`, `addDemographics()`, `addCohortIntersectFlag()`, `addCohortIntersectCount()`, `addCohortIntersectDate()`, `addDrugUse()`
`summarise*`	`summarised_result`	None	`summariseCharacteristics()`, `summariseClinicalRecords()`, `summariseDrugUtilisation()`, `summariseCohortAttrition()`, `summariseCohortCount()`
`estimate*`	`summarised_result`	None	`estimateIncidence()`, `estimatePointPrevalence()`, `estimatePeriodPrevalence()`
`plot*`	`ggplot2` object	None	`plotRecordCount()`, `plotIncidence()`, `plotPrevalence()`, `plotCohortAttrition()`
`table*`	Formatted table (`gt`, `flextable`, etc.)	None	`tableCharacteristics()`, `tableClinicalRecords()`, `tableOmopSnapshot()`, `tableCohortCount()`

Prefix	Returns	Side effects	Representative examples
`mock*`	`cdm_reference` (in-memory)	None	`mockCohortConstructor()`, `mockPatientProfiles()`, `mockDrugUtilisation()`, `mockOmopSketch()`, `mockIncidencePrevalence()`
`get*`	Plain R object	None	`getDrugIngredientCodes()`, `getVocabVersion()`
`new*`	New instance of a class	None	`newCodelist()`, `newSummarisedResult()`, `newCdmSource()`
`compute*`	Materialised table	Writes temp table to DB	`computeTable()` (internal convention in PatientProfiles)

Note

`estimate*` is used in `IncidencePrevalence` rather than `summarise*` because incidence and prevalence are computed statistics with a specific analytic meaning (rates and proportions over time), not summaries in the general sense. New packages that compute similar analytic estimates should consider whether `estimate*` or `summarise*` better describes the result.

Note

`require*` is specific to `CohortConstructor` and means "apply this inclusion or exclusion criterion and update the cohort attrition". Do not use `require*` for functions that merely check or validate inputs — use `validate*` or `assert*` for those.

7.2.2. The `*CohortSet` suffix

Functions that generate multiple cohorts at once and return a full `cohort_table` in the CDM use the `*CohortSet` suffix:

```
generateConceptCohortSet(cdm, conceptSet = ..., name = "my_cohort")
generateIngredientCohortSet(cdm, ingredient = "acetaminophen", name = "dus_cohort")
generateDenominatorCohortSet(cdm, name = "denominator", ageGroup = list(...))
```

Functions that return a single cohort or modify an existing table typically do not use this suffix.

7.2.3. Noun component

The noun should describe what the function operates on or returns clearly enough that users can understand it without reading the documentation:

- `summariseClinicalRecords` — summarises records in clinical OMOP tables
- `addCohortIntersectFlag` — adds a flag column indicating cohort intersection
- `requireConceptIntersect` — requires intersection with a concept set

Avoid abbreviations unless they are standard in the domain (e.g. `Cdm`, `Omop`). Spell out words fully: `Incidence` not `Inc`, `Characteristics` not `Chars`.

7.2.4. `mock*` functions

Every package should provide at least one `mock*` function so that examples and tests can run without a real database. The naming convention is `mock{PackageName}()`:

```r
mockPatientProfiles()        # PatientProfiles
mockCohortConstructor()      # CohortConstructor
mockDrugUtilisation(numberIndividual = 100, source = "duckdb")
mockCohortCharacteristics()  # CohortCharacteristics
mockOmopSketch()             # OmopSketch
mockIncidencePrevalence(sampleSize = 1000)
```

Internally, these functions typically call `omock` helpers such as `mockCdmFromDataset()` or `mockCdmFromTables()`. The `mock*` function should return a `cdm_reference` containing whatever tables the package needs, and should support a `source` argument (`"duckdb"` or `"local"`) where appropriate.

Mock functions should be called `mockDisconnect(cdm)` at the end of examples to release the database connection — a convention from `PatientProfiles` now used widely.

7.3. Argument naming

7.3.1. Standard argument names

The following argument names are reserved. Use them whenever your function has an argument that matches the described role.

Argument	Type	Role
`cdm`	`cdm_reference`	CDM reference object. First argument if present.
`cohort`	`cohort_table`	Cohort table to operate on.

Argument	Type	Role
`cohortId`	`integer` or `NULL`	IDs of cohorts to include; `NULL` = all.
`conceptSet`	codelist / expression	Clinical concept set.
`name`	`character(1)`	Name of the output table written into the CDM.
`nameStyle`	`character(1)`	Glue template for naming output columns.
`targetCohortTable`	`character(1)`	Name of a cohort table to intersect with.
`targetCohortId`	`integer` or `NULL`	Cohort IDs within `targetCohortTable`.
`tableName`	`character(1)`	Name of an OMOP CDM clinical table.
`omopTableName`	`character(1)`	As `tableName`; used in OmopSketch-style functions.
`indexDate`	`character(1)`	Column name to use as the index date.
`strata`	`list` of `character` vectors	Stratification variable sets.
`ageGroup`	named `list` or `NULL`	Age group definitions, e.g. `list(c(0, 17), c(18, 64))`.
`sex`	`character`	Sex filter or stratification: `"Male"`, `"Female"`, `"Both"`.
`window`	length-2 integer vector or named `list`	Time window(s) relative to index date.
`intersections`	integer or integer vector of length 2	Number of required intersections.

Argument	Type	Role
`gapEra`	`integer(1)`	Days between records below which they are merged into one era.
`minCellCount`	`integer(1)`	Minimum cell count for suppression. Default `5`.
`type`	`character(1)`	Output format for `table*`: `"gt"`, `"flextable"`, `"tibble"`, `"datatable"`.
`facet`	`character` or formula	Columns to facet by in `plot*` functions.
`colour`	`character`	Columns to colour by in `plot*` functions.

7.3.2. Argument order

Arguments should appear in this order:

1. `cdm` (if the function takes the CDM directly)
2. `cohort` (if the function takes a cohort table)
3. `cohortId`
4. Content arguments (what to compute, concept sets, windows, etc.)
5. Modifier arguments (flags and options that adjust behaviour)
6. `name` (near the end, since it is often left at its default)

```
# cdm first, then content, then name
generateIngredientCohortSet(
  cdm = cdm,
  name = "dus_cohort",
```

```
  ingredient = "acetaminophen",
  gapEra = 7
)

# cohort first, then targetCohortTable, then window, then name
requireCohortIntersect(
  cohort = cdm$fractures,
  targetCohortTable = "gibleed",
  intersections = c(1, Inf),
  window = c(-Inf, 0),
  name = "fractures_with_gi_bleed"
)
```

7.3.3. The `nameStyle` argument

`add*` functions in `PatientProfiles` that produce multiple columns use a `nameStyle` argument — a glue-style template that controls how the new column names are constructed. Available template variables depend on the function but commonly include `{cohort_name}`, `{window_name}`, `{concept_name}`, and `{value}`.

```
# Default nameStyle - column is named automatically
addCohortIntersectFlag(cohort, targetCohortTable = "cohort2", window = c(-Inf, -1))

# Custom nameStyle - single column named "prior_infection"
addCohortIntersectFlag(
  cohort,
  targetCohortTable = "cohort2",
  window = c(-Inf, -1),
  nameStyle = "prior_infection"
)
```

```r
# Multi-window nameStyle - columns like "prior_morphine", "future_aspirin"
addCohortIntersectCount(
  cohort,
  targetCohortTable = "drugs",
  window = list("prior" = c(-Inf, -1), "future" = c(1, Inf)),
  nameStyle = "{window_name}_{cohort_name}"
)
```

When your `add*` function can produce multiple columns, provide a sensible default `nameStyle` and document the available template variables in the function's roxygen entry.

7.4. Windows

Time windows are integer vectors of length 2: `c(start, end)`, where values are days relative to an index date (typically `cohort_start_date`).

- `0` means the index date itself.
- `-Inf` means no lower bound (any time before).
- `Inf` means no upper bound (any time after).

When multiple windows are needed, they are passed as a **named list**. The names become the window labels in `nameStyle` templates and in `additional_name`/`additional_level` columns of `summarised_result` objects:

```r
window = list(
  "short_term" = c(1, 30),
  "mid_term"   = c(31, 180),
  "long_term"  = c(181, 365)
)
```

7.5. Sex values

When sex is stored as a value — in `strata_level`, as a column value, or in cohort settings — the standard values are `"Male"`, `"Female"`, and `"Both"`. These use an initial capital. This convention is established in `IncidencePrevalence` and used consistently across the ecosystem.

7.6. Data values in `summarised_result`

String values stored inside `summarised_result` columns use `snake_case`. This applies to `variable_name`, `variable_level`, `estimate_name`, `result_type`, and values in `group_level` and `strata_level`:

```
# Typical variable_name values
"number_records"
"number_subjects"
"age"
"sex"

# Typical estimate_name values
"count"
"mean"
"sd"
"q25"
"median"
"q75"

# Typical result_type values
"summarised_characteristics"
"summarised_drug_utilisation"
"cohort_count"
```

7.7. Column naming in cohort tables

Columns added to cohort tables by `add*` functions use `snake_case`. The names are derived from the `nameStyle` template and typically combine a descriptor of the value type, a cohort or concept name, and a window name when multiple windows are used:

Pattern	Example column name
Single target, default style	`cohort_2_minf_to_m1`
Custom `nameStyle = "prior_infection"`	`prior_infection`
`nameStyle = "{window_name}_{cohort_name}"`	`prior_morphine`, `future_aspirin`
`addAge()`	`age`
`addSex()`	`sex`
`addPriorObservation()`	`prior_observation`

7.8. The `summarise` → `table/plot` pattern

All analytics packages follow the same two-step workflow:

1. A `summarise*` (or `estimate*`) function queries the CDM and returns a `summarised_result`.
2. A paired `table*` or `plot*` function formats that result for display.

Do not pipe `table*` or `plot*` functions directly onto `summarise*` calls in production code. The `summarised_result` object is the exportable artefact — you may want to save it, combine it with results from other databases, or suppress it before rendering.

```
# Correct workflow
result <- summariseCharacteristics(cdm$my_cohort)
result |> tableCharacteristics(type = "gt")

# Also fine for quick exploration, but not recommended in scripts
# cdm$my_cohort |>
#   summariseCharacteristics() |>
#   tableCharacteristics()
```

7.9. Inconsistencies to be aware of

The ecosystem has grown over several years, and a small number of inconsistencies currently exist:

- **`omopTableName` vs `tableName`**: `OmopSketch` uses `omopTableName` for the name of a CDM clinical table; other packages use `tableName`. New packages should prefer `tableName` unless interoperating closely with `OmopSketch` patterns.
- **`sex` argument type**: Some older functions accept `sex = TRUE` (logical, meaning "stratify by sex") while newer ones accept `sex = c("Male", "Female", "Both")` (character, meaning "filter to these groups"). Check the package you are building on and match its convention; prefer the character form in new code.
- **`estimate*` vs `summarise*`**: `IncidencePrevalence` uses `estimate*` for its main output functions. Other packages use `summarise*`. Both conventions will coexist; choose the one that best describes what your function computes.
- **`mockDisconnect()` location**: The `mockDisconnect()` helper is exported from `PatientProfiles`. Packages that use it in examples need `PatientProfiles` in `Suggests`.

When you find a convention that is not documented here or that appears applied inconsistently, please open an issue on the book repository.

8. Error messages and input validation

Giving users informative error messages is one of the most important things you can do to make a package pleasant to use. A cryptic error from deep inside a dplyr pipeline — or worse, a silent wrong result — is much harder to debug than a clear message at the top of the function that says exactly what was wrong with the input.

The rule of thumb is: **validate at the boundary, trust inside**. Check all inputs at the very start of each exported function, before any work is done. Once those checks pass, do not re-check the same things in internal helpers.

That said, do not overdo it. Avoid checking things that are guaranteed by earlier checks, and avoid checks that cost more than the function itself. In general, check that each argument has the expected type, that it has the expected length, and that any logical constraints between arguments are satisfied.

8.1. The two kinds of validation helpers in `omopgenerics`

`omopgenerics` provides two families of validation helpers that together cover almost everything you need.

`validate*` functions are for arguments that appear consistently across many packages — `cdm`, `cohort`, `cohortId`, `conceptSet`, `window`, `ageGroup`, `strata`, `name`, `nameStyle`, and `result`. These functions do more than just check: they also **coerce and normalise** the input to a canonical form. You must assign their return value back to the variable, because the cleaned-up version is what the rest of your function should use.

`assert*` functions are for simple type and constraint checks. They do not transform the input — they either pass silently or throw an error. Use these for arguments that do not have a dedicated `validate*` function.

8.2. `validate*` functions

The following `validate*` functions are available in `omopgenerics`. Import them with `@importFrom omopgenerics validateCdmArgument` (and so on) rather than calling them with `omopgenerics::`.

Function	Validates and normalises
`validateCdmArgument(cdm)`	CDM reference: checks class, optionally checks required tables
`validateCohortArgument(cohort)`	Cohort table: checks class, required columns, attributes
`validateCohortIdArgument(cohortId, cohort)`	Cohort IDs: resolves `NULL` to all IDs, checks IDs exist in cohort
`validateConceptSetArgument(conceptSet)`	Concept set: accepts codelist, codelist_with_details, or concept_set_expression
`validateWindowArgument(window)`	Window: accepts vector or named list, always returns named list

Function	Validates and normalises
`validateAgeGroupArgument(ageGroup)`	Age groups: accepts several input forms, always returns named list of named intervals
`validateStrataArgument(strata, cohort)`	Strata: checks that all columns named in strata exist in the cohort
`validateNameArgument(name, cdm)`	Table name: checks it is a valid identifier, optionally checks it doesn't already exist
`validateNameStyle(nameStyle, ...)`	Name style template: checks it is a valid glue template
`validateResultArgument(result)`	Summarised result: checks class and required columns

8.2.1. Normalisation is the point

The most important property of `validate*` functions is that they accept several reasonable input forms and always return a single canonical form. This means the rest of your function only has to handle one form, which keeps the logic clean.

The `ageGroup` argument illustrates this well. Users can supply it in several ways:

```
library(omopgenerics, warn.conflicts = FALSE)

# Simple unnamed intervals
ageGroup <- validateAgeGroupArgument(ageGroup = list(c(0, 17), c(18, 64)))
ageGroup
```

```
$age_group
$age_group$`0 to 17`
```

```
[1]  0 17

$age_group$`18 to 64`
[1] 18 64
```

```r
# Complex named multi-column form
ageGroup <- validateAgeGroupArgument(ageGroup = list(
  age_at_start = list("child" = c(0, 17), "adult" = c(18, 64)),
  age_at_end   = list(c(0, 49), c(50, Inf))
))
ageGroup
```

```
$age_at_start
$age_at_start$child
[1]  0 17

$age_at_start$adult
[1] 18 64


$age_at_end
$age_at_end$`0 to 49`
[1]  0 49

$age_at_end$`50 or above`
[1]  50 Inf
```

Whatever you pass in, the output is always a named list of named intervals. The function also throws clear errors if the input is malformed:

```r
# Overlapping age groups when only one column is allowed
validateAgeGroupArgument(
```

```
  ageGroup = list(group1 = list(c(0, 19), c(20, Inf)), group2 = list(c(0, Inf))),
  multipleAgeGroup = FALSE
)
```

```
Error:
! Multiple age group are not allowed
```

```
# Negative ages
validateAgeGroupArgument(ageGroup = list(c(-5, 19), c(20, Inf)))
```

```
Error in `purrr::map()`:
i In index: 1.
Caused by error:
! Elements of `ageGroup` argument must be greater or equal to "0".
```

```
# NULL not allowed
validateAgeGroupArgument(ageGroup = NULL, null = FALSE)
```

```
Error:
! `ageGroup` argument can not be NULL.
```

The same principle applies to `validateWindowArgument()`, which accepts either a plain `c(0, Inf)` vector or a named list of windows, and always returns a named list.

8.2.2. The {{ syntax for `cohortId`

`validateCohortIdArgument()` supports tidyselect semantics, so users can pass things like `starts_with("fracture")` to select cohort IDs by name. To make this work, you must wrap the argument in double curly braces `{{` when you pass it to the validator:

```
cohortId <- validateCohortIdArgument(cohortId = {{cohortId}}, cohort = cohort)
```

Without `{{`, tidyselect expressions will not be evaluated correctly.

8.3. `assert*` functions

Use `assert*` functions for arguments that do not have a dedicated `validate*` function. Unlike `validate*`, these functions return the input invisibly and are used purely for their side effect of throwing an error.

Function	Checks
`assertCharacter(x, ...)`	`x` is a character vector
`assertNumeric(x, ...)`	`x` is numeric
`assertLogical(x, ...)`	`x` is logical
`assertDate(x, ...)`	`x` is a Date
`assertList(x, ...)`	`x` is a list
`assertClass(x, class, ...)`	`x` has the specified class(es)
`assertChoice(x, choices, ...)`	`x` is one of the allowed values
`assertTable(x, ...)`	`x` is a table (data frame or `tbl_sql`)
`assertTrue(expr, ...)`	The expression evaluates to `TRUE`

All of these accept common modifiers as arguments:

Modifier	Type	Meaning
`length`	`integer` or `NULL`	Required length; `NULL` skips the check
`null`	`logical`	Whether `NULL` is a valid input (default `FALSE`)

Modifier	Type	Meaning
`na`	`logical`	Whether `NA` values are allowed (default `FALSE`)
`named`	`logical`	Whether elements must be named
`unique`	`logical`	Whether elements must be unique
`min`	`numeric`	Minimum value (for `assertNumeric`)
`max`	`numeric`	Maximum value (for `assertNumeric`)
`integerish`	`logical`	Whether numeric must be whole numbers (for `assertNumeric`)
`minNchar`	`integer`	Minimum number of characters per element (for `assertCharacter`)
`call`	call	Passed to the `cli` error message for better call context

8.4. Putting it together: a validation block

Here is the recommended pattern. Validation comes first, before any computation. `validate*` results are always reassigned. `assert*` calls are written as statements.

```
myFunction <- function(cohort, cohortId = NULL, window = c(0, Inf), overlap = FALSE)
  # validate and normalise
  cohort   <- omopgenerics::validateCohortArgument(cohort = cohort)
  cohortId <- omopgenerics::validateCohortIdArgument(cohortId = {{cohortId}}, cohort
  window   <- omopgenerics::validateWindowArgument(window = window)

  # simple type checks
  omopgenerics::assertLogical(overlap, length = 1)

  # business logic checks
  if (overlap && any(sapply(window, function(w) diff(w) > 365))) {
    cli::cli_abort(c(
      "x" = "Windows longer than 365 days are not supported when {.var overlap} is {
    ))
  }

  # ... function body
}
```

A second example with a CDM reference:

```
myFunction2 <- function(cdm, conceptSet, days = 180L, startDate = NULL, overlap = TR
  # validate and normalise
  cdm        <- omopgenerics::validateCdmArgument(cdm = cdm)
  conceptSet <- omopgenerics::validateConceptSetArgument(conceptSet = conceptSet)

  # simple type checks
  omopgenerics::assertNumeric(days, integerish = TRUE, min = 0, length = 1)
  omopgenerics::assertDate(startDate, length = 1, null = TRUE)
  omopgenerics::assertLogical(overlap, length = 1)

  # business logic checks
  if (overlap && days > 365) {
```

```
    cli::cli_abort(c(
      "x" = "{.var days} cannot be >= 365 when {.var overlap} is {.val TRUE}."
    ))
  }

  # ... function body
}
```

8.5. Writing good error messages with `cli`

Custom errors and warnings should use the **cli** package, which is already a dependency of `omopgenerics`. Do not use `stop()` or `warning()`.

```
# Error
cli::cli_abort(c(
  "x" = "Argument {.var days} must be a positive integer.",
  "i" = "You supplied {.val {days}}."
))

# Warning
cli::cli_warn(c(
  "!" = "Cohort {.val {name}} already exists in the CDM and will be overwritten."
))

# Informational message
cli::cli_inform(c(
  "i" = "Computing {length(cohortId)} cohort{?s}."
))
```

A few conventions for error messages:

- Use `{.var argument_name}` to refer to an argument by name.

- Use `{.val {value}}` to show the actual value that was supplied.
- Use `{.fn function_name}` to refer to a function.
- Use `{.cls class_name}` to refer to a class.
- Use named elements `"x"` for errors, `"!"` for warnings, `"i"` for hints or additional information.
- Write error messages in the present tense from the user's perspective: "must be", "cannot be", "is not" rather than "expected", "got".

8.6. What to validate and what to skip

Validate arguments that users are likely to pass in many ways or get wrong:

- Arguments with a standard `validate*` function — always validate these.
- Type-sensitive arguments: anything the rest of the function passes directly to dplyr or database operations.
- Arguments with constraints relative to each other (e.g. `days` must be positive when `overlap` is `TRUE`).

Do not validate:

- Internal function arguments that are never user-facing.
- Things already guaranteed by a `validate*` call (e.g. do not re-check that `cohortId` is numeric after calling `validateCohortIdArgument`).
- Constants you define yourself inside the function.

8.7. Summary

Validating arguments is one of the most impactful things you can do for users — it turns opaque database errors into clear messages that point directly to the problem. `omopgenerics` makes this straightforward: use

`validate*` for the standard OMOP arguments (and always reassign the result), use `assert*` for simple type checks, and use `cli` for any custom messages. A complete validation block for a typical function takes four or five lines and requires no custom logic.

9. Function outputs

9.1. The `summarised_result` object

The `summarised_result` is the standard result format returned by all analytics functions in the ecosystem. It is an S3 class built on top of a tibble, defined in `omopgenerics`. Every package that produces estimates — prevalence, drug use, cohort characteristics, survival — returns a `summarised_result`. This standardisation is what allows `visOmopResults` and `OmopViewer` to work uniformly across the whole ecosystem without any package-specific code.

9.1.1. Structure

A `summarised_result` is a tibble with exactly 13 compulsory columns:

Column	Type	NA allowed	Description
`result_id`	integer	No	Groups rows sharing the same result settings.
`cdm_name`	character	No	Name of the CDM instance used.

Column	Type	NA allowed	Description
`group_name`	character	No	Name side of the group name–level pair.
`group_level`	character	No	Level side of the group name–level pair.
`strata_name`	character	No	Name side of the strata name–level pair.
`strata_level`	character	No	Level side of the strata name–level pair.
`variable_name`	character	No	Name of the variable being estimated.
`variable_level`	character	Yes	Sub-level of the variable (e.g. a category).
`estimate_name`	character	No	Name of the estimate (snake_case).
`estimate_type`	character	No	Type of the estimate value (see below).
`estimate_value`	character	No	Value of the estimate, always stored as character.
`additional_name`	character	No	Name side of the additional name–level pair.

Column	Type	NA allowed	Description
`additional_level`	character	No	Level side of the additional name–level pair.

The `estimate_type` column must be one of the values returned by `omopgenerics::estimateTypeChoices()`: `"numeric"`, `"integer"`, `"date"`, `"character"`, `"proportion"`, `"percentage"`, `"logical"`.

9.1.1.1. Name–level pairs

Three pairs of columns — `group_name`/`group_level`, `strata_name`/`strata_level`, `additional_name`/`additional_level` — follow the same encoding. When a result applies to multiple dimensions simultaneously, the column names are concatenated with `&&&` and the values likewise:

```
group_name:  "cohort_name &&& sex"
group_level: "asthma_cohort &&& Female"
```

- **group**: high-level identifiers, typically cohort name or codelist name.
- **strata**: stratification dimensions such as age group or sex.
- **additional**: any further dimensions that don't fit in group or strata (e.g. time window, analysis type).

When a result is not stratified, these columns should contain `"overall"` / `"overall"`.

9.1.2. Settings attribute

Every `summarised_result` has a `settings` attribute — a tibble with one row per `result_id` — that stores metadata about how the results were generated. Three settings columns are compulsory:

Setting	Description
`result_type`	A short snake_case string identifying the type of analysis (e.g. `"summarised_characteristics"`). Must be unique within a package.
`package_name`	The name of the package that generated the result.
`package_version`	The version of that package.

Additional settings columns store analysis-specific parameters (e.g. `strata`, `ageGroup`, `window`). A `min_cell_count` column is always present, defaulting to `"0"`.

9.1.3. How to create a `summarised_result`

The standard pattern is to build a plain tibble with the 13 required columns, then convert it using `omopgenerics::newSummarisedResult()`:

```
library(omopgenerics)

result <- tibble::tibble(
  result_id       = 1L,
  cdm_name        = cdmName(cdm),
  group_name      = "cohort_name",
  group_level     = "my_cohort",
  strata_name     = "overall",
  strata_level    = "overall",
  variable_name   = "age",
  variable_level  = NA_character_,
  estimate_name   = "mean",
  estimate_type   = "numeric",
```

```
  estimate_value  = as.character(mean_age),
  additional_name = "overall",
  additional_level= "overall"
) |>
  newSummarisedResult(
    settings = tibble::tibble(
      result_id       = 1L,
      result_type     = "my_result_type",
      package_name    = "myPackage",
      package_version = as.character(packageVersion("myPackage"))
    )
  )
```

In practice you will produce many rows at once. The recommended internal workflow is:

1. Compute a tidy intermediate tibble (one row per subject / cohort / stratum / estimate).
2. Use `visOmopResults::uniteNameLevel()` or `omopgenerics` helpers to encode name–level pairs.
3. Call `newSummarisedResult()` with the appropriate settings.

9.1.4. Using the tidy format internally

The `tidy()` method converts a `summarised_result` back to a wide, human-readable format:

```
result |> tidy()
```

This is useful for inspecting results during development, and can also be used internally if your function needs to manipulate results before returning them. However, always return a `summarised_result` from exported functions — never a tidy tibble.

9.1.5. Suppression

If your function produces counts of patients or records, it must support cell suppression. The convention is to expose a `minCellCount` argument (default `5`) and apply suppression before returning:

```
result <- result |> suppress(minCellCount = minCellCount)
```

`suppress()` replaces estimate values with `NA` where the count is below the threshold, and records the suppression threshold in the settings.

If your result contains a count of subjects (i.e. unique `subject_id` values), the estimate should be named `count_subjects` or `n_subjects` so that suppression is applied correctly.

9.1.6. One `result_type` per function

Each exported function that returns a `summarised_result` should produce exactly one `result_type`. Do not mix different analyses in a single `summarised_result` returned from one function — this makes it impossible for downstream tools to filter and process results by type.

9.1.7. Registering a `result_type`

Result types should be registered in your package so that `visOmopResults` and `OmopViewer` can discover them. Add a call to `omopgenerics::registerResultType()` in your package's `.onLoad()` or document it in the package metadata. See the `omopgenerics` documentation for the current registration API.

9.2. Cohort outputs

Functions that return a modified cohort (rather than a summary) should return a `cohort_table`. The attrition log must be updated for every filtering step using `recordCohortAttrition()`. See Chapter 5 for the `cohort_table` class description.

9.3. Further reading

- `omopgenerics` summarised_result documentation
- `visOmopResults` package

10. Visualisations

All visualisation in the ecosystem is built on top of `visOmopResults`. This package provides a consistent API for transforming `summarised_result` objects into formatted tables and `ggplot2` plots. By using it rather than writing bespoke formatting code, your package inherits consistent output across all result types and avoids duplicating logic that is already maintained centrally.

This chapter explains the conventions for adding `table*` and `plot*` functions to your package.

10.1. The `visOmopResults` API

`visOmopResults` exposes two main families of functions:

- **`visOmopTable()`** and its wrappers produce formatted tables. The output format (gt, flextable, tibble, or datatable) is controlled by a `type` argument.
- **`visOmopPlot()`** and specific plot helpers produce `ggplot2` objects that can be further customised.

Most packages do not call these low-level functions directly. Instead they implement thin `table*` and `plot*` wrapper functions that call into `visOmopResults` with sensible defaults.

10.2. Adding `table*` functions

Every `summarise*` function should have a corresponding `table*` function that formats its output for display. The convention is:

```
tableMyResult <- function(result, type = "gt", ...) {
  # optionally filter/reshape result before passing to visOmopResults
  visOmopResults::visOmopTable(
    result = result,
    type   = type,
    ...
  )
}
```

Key points:

- The first argument is always a `summarised_result`, typically the output of the matching `summarise*` function.
- `type` defaults to `"gt"` in function signatures but **use `type = "flextable"` in README examples** — flextable renders better in GitHub Markdown.
- Pass `...` through to `visOmopResults::visOmopTable()` to expose column renaming, formatting, and other options without duplicating them in your own function signature.
- Validate the `result` argument using `omopgenerics::validateResultArgument()`.

A minimal example:

```
#' Format summarised characteristics as a table
#'
#' @param result A \code{summarised_result} object as returned by
#'   \code{summariseCharacteristics()}.
#' @param type Output format. One of \code{"gt"} (default), \code{"flextable"},
```

```
#'   \code{"tibble"}, or \code{"datatable"}.
#' @param ... Additional arguments passed to
#'   \code{\link[visOmopResults]{visOmopTable}}.
#'
#' @return A formatted table object of the class determined by \code{type}.
#' @export
tableCharacteristics <- function(result, type = "gt", ...) {
  result <- omopgenerics::validateResultArgument(result)
  visOmopResults::visOmopTable(result = result, type = type, ...)
}
```

10.3. Adding `plot*` functions

Similarly, `plot*` functions wrap `visOmopResults` plot helpers:

```
#' Plot incidence estimates
#'
#' @param result A \code{summarised_result} object as returned by
#'   \code{estimateIncidence()}.
#' @param ... Additional arguments passed to the underlying plot function.
#'
#' @return A \code{ggplot2} object.
#' @export
plotIncidence <- function(result, ...) {
  result <- omopgenerics::validateResultArgument(result)
  visOmopResults::plotEstimates(result = result, ...)
}
```

Because `plot*` functions return `ggplot2` objects, users can add layers, change themes, and save the plot using standard ggplot2 idioms. Do not apply a custom theme inside your package's plot functions — leave theming to the user.

10.4. Checking what is already available

Before writing any visualisation code, check the visOmopResults function reference. The package is actively extended, and the function you need may already exist. Duplicating it in your own package creates a maintenance burden and means your users get a subtly different experience.

10.5. What not to do

- Do not return raw `gt` or `flextable` objects directly from `table*` functions — always go through `visOmopResults::visOmopTable()` so that the `type` argument works consistently.
- Do not hard-code column names or formatting inside your package — use the `visOmopResults` API so that changes to the formatting logic can be made in one place.
- Do not depend on `ggplot2` directly unless your package needs to construct plots from scratch (i.e. not from a `summarised_result`). For `summarised_result`-based plots, use `visOmopResults`.

10.6. Suggesting vs importing `visOmopResults`

For most packages, `visOmopResults` should be listed under `Imports` rather than `Suggests`, because the `table*` and `plot*` functions are part of the package's exported API and will fail if `visOmopResults` is not installed. If your package only uses `visOmopResults` in vignettes or optional code paths, `Suggests` is appropriate.

11. Styling your code

Consistent code style across the ecosystem means that any contributor can read and navigate any package without having to mentally re-parse a different formatting convention. This chapter describes the style rules used and the tools that enforce them.

11.1. Code style

The ecosystem follows the tidyverse style guide with one important modification: **function and argument names use camelCase**, not snake_case. This applies to all exported functions, unexported functions, and arguments. Object names inside function bodies should also use camelCase.

```
# Correct
summariseCohortOverlap <- function(cohort, cohortId = NULL, minCellCount = 5) {
  resultData <- computeOverlap(cohort, cohortId)
  resultData |> formatOutput(minCellCount = minCellCount)
}

# Incorrect - snake_case names
summarise_cohort_overlap <- function(cohort, cohort_id = NULL, min_cell_count = 5) {
  result_data <- compute_overlap(cohort, cohort_id)
}
```

The rationale for camelCase over snake_case is historical consistency: the earliest packages in the ecosystem (including `omopgenerics` and `PatientProfiles`) used camelCase, and changing convention mid-ecosystem would introduce more confusion than it resolves.

11.2. Linting

All packages in the ecosystem use `lintr` to enforce style. The standard configuration is:

```
lintr::lint_package(
  linters = lintr::linters_with_defaults(
    lintr::object_name_linter(styles = "camelCase")
  )
)
```

This checks all camelCase naming conventions, line length, spacing, and other tidyverse style rules, with the camelCase override for names.

You should run `lint_package()` before opening any pull request. The GitHub Actions CI workflow (see **?@sec-automatic-checks**) runs linting automatically on every push.

To add lintr configuration to your package so that the linter settings are stored in the repository:

```
usethis::use_lintr()
```

This creates a `.lintr` file at the root of your package. Edit it to match the ecosystem configuration:

```
linters: linters_with_defaults(
  object_name_linter(styles = "camelCase")
)
```

11.3. Formatting

Use `styler` to automatically format code. The standard command is:

```
styler::style_pkg()
```

> **!** Important
>
> When contributing to an existing package, **do not restyle code that you have not otherwise modified**. Reformatting unrelated lines makes diffs harder to review and is a common source of merge conflicts. Only restyle the lines you have actively changed.

11.4. Line length

The maximum line length is 80 characters. Long function calls should be broken across lines with one argument per line, with the closing parenthesis on its own line:

```
# Correct
result <- summariseCharacteristics(
  cohort = cdm$my_cohort,
  strata = list(c("sex"), c("age_group")),
  minCellCount = 5
)

# Avoid - too long for a single line
result <- summariseCharacteristics(cohort = cdm$my_cohort, strata = list(c("sex"), c
```

11.5. Pipes

Use the native pipe `|>` introduced in R 4.1. The `%>%` pipe from magrittr should not be used in new code. The native pipe is imported automatically and requires no additional dependency.

11.6. Assignment

Use `<-` for assignment, not `=`. Reserve `=` for argument passing inside function calls.

11.7. Spacing

- Put spaces around all binary operators (`<-`, `+`, `-`, `==`, `|>`, etc.).
- Put a space after commas but not before.
- Do not put spaces inside parentheses: `f(x, y)` not `f( x, y )`.

These rules are enforced by `lintr` and will be flagged automatically.

12. Reexporting from other packages

Re-exporting a function from another package means making it available as if it were part of your own package. When a user attaches your package with `library(myPackage)`, re-exported functions appear in the namespace and can be called without a package prefix.

This is a feature that should be used sparingly and deliberately. This chapter explains when re-exporting is appropriate, how to do it, and what the implications are.

12.1. When to re-export

Re-exporting is appropriate in a small number of situations:

You are implementing a generic defined in `omopgenerics`. If `omopgenerics` defines a generic (e.g. `settings()`, `cohortCount()`, `attrition()`) and your package provides a method for it, you should re-export the generic so users can call it without loading `omopgenerics` explicitly. This is the most common legitimate use of re-exports in this ecosystem.

Your package is a high-level wrapper. If your package intentionally acts as a one-stop interface (e.g. a meta-package or a workflow package) and re-exports functions to save users from loading multiple packages, that is reasonable — but document the choice explicitly.

You re-export the pipe. Re-exporting `|>` is not necessary since R 4.1, as it is a base R operator. Do not re-export `%>%` from magrittr.

12.2. When not to re-export

- Do not re-export functions simply because your package depends on them internally. That they are in `Imports` does not mean they belong in your namespace.
- Do not re-export large swaths of another package's API. This creates namespace pollution for your users and tightly couples your package's public interface to the internals of another package.
- Do not re-export functions that are likely to change in upstream packages — every breaking change upstream becomes a breaking change in your package too.

12.3. How to re-export

Use the `@importFrom` and `@export` roxygen2 tags together:

```
#' @importFrom omopgenerics settings
#' @export
omopgenerics::settings
```

This pattern tells roxygen2 to import the function from `omopgenerics` and re-export it from your package's namespace. After running `devtools::document()`, the function will appear in your `NAMESPACE` file with both `importFrom` and `export` entries.

Alternatively, for generics you implement, you can use the `@rdname` tag to link your method to the upstream documentation:

```r
#' @rdname omopgenerics::settings
#' @export
settings.myClass <- function(x, ...) {
  # your implementation
}
```

12.4. Implications of re-exporting

Re-exporting creates a **public commitment**. If `omopgenerics` changes or removes `settings()`, your package will break at the namespace level — not just internally. Users loading your package will see `settings()` in your namespace, and if it disappears, they will blame your package.

For this reason, only re-export functions from packages whose stability you trust and whose interfaces are unlikely to change in breaking ways. `omopgenerics` generics are the safest candidates because the package is deliberately stable and versioned.

12.5. Checking your exports

After calling `devtools::document()`, inspect your `NAMESPACE` file to confirm that only the functions you intended to export appear there. An unexpectedly large export list is a sign that re-exports have been added too liberally.

You can also use:

```r
# List all exports from your package
getNamespaceExports("myPackage")
```

Part III.

Contributing documentation

Documentation is not optional. A well-documented package is one that people can actually use, contribute to, and trust. For packages in the Tidy R OMOP CDM ecosystem — which are tools for epidemiological research — clear documentation can directly affect the quality of scientific work that depends on them.

This part of the book covers all the layers of documentation that a package should have, from inline code comments to a full pkgdown website. Each layer serves a different audience and a different purpose, and all of them matter.

The chapters are organised as follows:

- Documenting code covers inline comments: when to write them, what they should say, and how to use them to communicate intent to future contributors (including your future self).
- Documenting functions covers roxygen2 documentation: how to write titles, descriptions, parameter documentation, and examples that genuinely help users understand and use your functions.
- README covers the package README: the first thing a new user will read. It should communicate the package's purpose clearly and include working examples that demonstrate its value.
- Vignettes covers long-form documentation: how to write vignettes that walk users through realistic use cases and help them understand not just individual functions but how to compose them into a complete analysis.
- Documentation website covers how to set up and customise a pkgdown website, organise the reference index, and deploy the site automatically via GitHub Actions.

13. Documenting code

Documenting code it is also important, after some weeks we will probably remember why you added some lines of code or what was needed next. Adding notes for yourself or the rest of developers it is always a good idea.

13.1. When to document?

In general not all the lines of code need to be documented, a very simple example can be:

```
y <- x + 1
```

You do not need a comment to know that the value of `y` is going to be `x` plus one unit. But maybe you need to document why you did that for example when we calculate duration it is `end - start + 1` as we want to calculate the number of days (start and end are included) and not the difference (`end - start`). This might not be obvious at first glance and giving a clue to you in the future or any other developer will save you time:

```
# end and start are included in duration, we want number of days not difference
duration <- end - start + 1
```

14. Documenting functions

We use the roxygen2 package for documenting our functions. Using this package will help us to write the .Rd files in the man directory which is what will be rendered when users call `?my_function` or `help("my_function")` and peruse a package website.

Above each exported function we'll create a roxygen **block**, with lines starting with `#'`. The general structure is shown below and we'll go through each in turn.

```
#' Title
#'
#' @description
#'
#' @param x
#' @param y
#'
#' @returns
#' @export
#'
#' @examples
```

14.1. Title

We want to provide a concise yet informative title for our function. The title should typically be written in sentence case but without any full stop

at the end.

If we run `help(package = "mypackage")` we can see all the functions and their titles for a package. From this we can quickly see how informative our titles are.

14.2. Description

Every function should have a desription. This could be a sentence or two for a simple function, or multiple paragraphs for a more complex function. The description should summarise the purpose of the function and explain any internal assumptions or decisions made to implement it.

When writing the description we can use markdown commands, for example to add bullet points, italics, etc. This will often help make longer desriptions more readable.

14.2.1. lifecycle badges

14.3. Parameters

After our description of the function as a whole, we will then describe each of the inputs to the function. In general we should be able to describe an argument in a few sentences. We should use them to tell the user what is allowed as an input and then what the input will be used for.

If we're working on a package where we have multiple arguments with the same argument we can document arguments just once. We can create a helper like below for parameter x.

```
#' Helper for consistent documentation of `x`.
#'
#' @param x
#'
#' @name xDoc
#' @keywords internal
```

We can then reuse this documentation across the package by using `@inheritParams`

```
#' Title
#'
#' @description
#'
#' @inheritParams xDoc
#' @param y
#'
#' @returns
#' @export
#'
#' @examples
```

14.4. Returns

We must also document what the function will return. For example, we can tell a user that our function will return a tibble with a certain set of columns. If our function has side-effects then we chould also document these.

14.5. Export

By including the `@export` tag our documentation will be generated as an `.Rd` file and users will see it in the package website, etc. If instead we want to document an internal function then we can replace this with `@noRd`.

14.6. Examples

Last but by no means least are a set of examples where we provide R code that shows how the function can be used in practice. These examples are quite possible the most important piece of documentation, as it is many users will look at these before reading any of the above.

Typically we should try to give a number of informative examples that show users how the function can be used and provides some intuition on the impact of different arguments.

15. README

The `README.md` is a very important part of an R package. It is probably the first thing the users will read once they decide to try out our package. The `README` must contain the goal of the package and clear examples so users can start using the package. Now we will so how to create it and how to populate it.

15.1. Create the `.Rmd` file

In general we will populate our `README.md` using a rmarkdown file (`README.Rmd`) if you don't have it created you can create it with:

```
library(usethis)
use_readme_rmd()
```

this will create the `README.Rmd` file that generates the `README.md`. To later build the `README.md` you can use the following command:

```
library(devtools)
build_readme()
```

If you are new to *rmakdown* probable this introduction will be useful: Introduction to R Markdown.

> **`README.md` and `README.Rmd`**
>
> Note that for cran/website/github only the `README.md` exists, so any change that you do to the `README.Rmd` file wont affect the 'real' README.md file till you build it. Make sure to build the `.md` file every time that you modify the `.Rmd`.

15.2. Content

A good `README.md` file must contain the following sections:

15.2.1. Badges

Badges are quite useful to display at the top of your README, they contain information of the package in a synthetic and consistent way. The badges are generated from `usethis` functions. You will have to manually copy the badge that will be printed in the console in the desired README section, usually at the top:

```
<!-- badges: start -->
PASTE BADGES HERE
<!-- badges: end -->
```

lifecycle status The development status badge can provide the user the development status of the package. We would usually use "experimental" for packages still under development (usually version $< 1.0.0$) and "stable" for stable packages, note if you add the stable badge this sets some expectations about the package like backwards compatibility. A lifecycle badge is highly recommended.

```
use_lifecycle_badge("experimental")
use_lifecycle_badge("stable")
```

cran version The cran badge informs about the current version in cran (if submitted to cran). This badge is also highly recommended (even before submitting to cran).

```
use_cran_badge()
```

build status The badge indicates if the last checks ran in your package passed or not. This badge should also be green otherwise this means the continuous integration is broken. If at some point actions break in main, fix it as soon as possible. This badge is also highly recommended. The badge is generated when GitHub Actions are set.

```
[![Build Status](https://github.com/{user/organisation}/{package_name}/workflows/R-
CMD-check/badge.svg)](https://github.com/{user/organisation}/{package_name}/actions?
CMD-check)
```

test coverage

cran downloads

15.2.2. Tested sources

If your packages is designed to work with different cdm sources (e.g. duckdb, local, postgres, sql server, …) we would encourage you to include the different sources you actively test against. We will later see the Testing multiple DBMS chapter where we explain how to do this automatically using GitHub actions. A good display to show the tested sources would be:

```
| Source | Driver | CDM reference | Status |
|------------------|------------------|------------------|------------------|
| Local R dataframe | N/A | `omopgenerics::cdmFromTables()` | [![](https://github.co
| In-memory duckdb datatabase | duckdb | `CDMConnector::cdmFromCon()` | [![](https:/
| Postgres database | RPostgres | `CDMConnector::cdmFromCon()` |  |
| Postgres database | DatabaseConnector | `CDMConnector::cdmFromCon()` |  |
| SQL Server database | odbc | `CDMConnector::cdmFromCon()` |  |
| SQL Server database | DatabaseConnector | `CDMConnector::cdmFromCon()` |  |
```

15.2.3. Goal of your package

Clearly state the goal of your package, do not extend much, usually 2/3 sentences are enough to describe what your package does.

15.2.4. How to install

Make sure to include information on how to install the package the current release from cran and the development version from GitHub. this section would usually look like:

```
# Installation

{pakage_name} can be installed from CRAN:

  ```r
  install.packages("{pakage_name}")
```

or

```
library(pak)
pkg_install("{pakage_name}")
```
```

If you prefere to use `pak`.

The development version can be installed from GitHub:

```
library(pak)
pkg_install("{user/organisation}/{package_name}")
```

```
### Simple examples

This is the most important section. Catch user attention, show how your package can

We also recommend to include diagrams on how some functions relate or how your packa

## Example

Here we created a simple example of how a good `README` would look like:


`<!-- quarto-file-metadata: eyJyZXNvdXJjZURpciI6ImRvY3VtZW50YXRpb24ifQ== -->`{=html}

```{=html}
<!-- quarto-file-metadata: eyJyZXNvdXJjZURpciI6ImRvY3VtZW50YXRpb24iLCJib29rSXRlbVR5c
```
```

16. Vignettes

17. Documentation website

Part IV.

Testing

Testing is a crucial part of R package development. Well-designed tests ensure that each function behaves as intended, protect the package from unexpected breakage when new features are added (new contributions should not cause existing tests to fail), and provide confidence when refactoring code (improving internal logic for efficiency should not alter the expected outputs). A comprehensive test suite also lowers the barrier for contributors: by making expected behaviour explicit, it becomes easier for others to understand how functions should work and to spot unintended changes.

This part of the book introduces good testing practices in the context of OMOP-based package development. It is organised into three chapters:

- Introduction to testing provides a high-level overview of testing in R using modern tools and conventions. This chapter summarises the key concepts presented in the Testing section of R Packages by Wickham and Bryan (https://r-pkgs.org/testing-basics.html).
- Testing in OMOP describes approaches for testing packages that interact with the OMOP CDM instances. In particular, it introduces the use of the omock R package [CITE], which enables the creation of synthetic OMOP data to validate package functionality without relying on a live database.
- Testing against multiple DBMS explains how to configure your package to run tests across multiple database management systems that host an OMOP CDM, ensuring that your code behaves consistently and reliably across different environments.

18. Introduction to testing

Testing is a fundamental part of developing reliable, maintainable R packages. A good test suite provides confidence that your functions behave as expected, prevents regressions when the code evolves, and acts as a form of living documentation of your package's intended behaviour. This chapter introduces the core ideas of testing in R, following tidyverse practices. It focuses on practical techniques you will use in every package: setting up tests, writing expectations, organising helpers and fixtures, deciding what to test, and documenting your tests clearly.

18.1. The Basics

R's testing ecosystem centres around the **testthat** package, supported by **usethis** for structuring test infrastructure and **devtools** for running tests during development. Together, these tools provide a simple and consistent workflow.

18.1.1. Setting up test infrastructure

The first step is to enable testing for your package:

```
library(usethis)
use_testthat()
```

This command:

- adds *testthat* to your package's `Suggests`
- creates a top-level file `tests/testthat.R`
- creates the directory `tests/testthat/` where test files will live

> **Parallel testing**
>
> Tests are run sequentially in alphabetical order of the files, but you can run those tests in parallel if you specify it:
>
> ```
> use_testthat(parallel = TRUE)
> ```
>
> Note that by default tests will be ran in 2 runners, but you can increase that number setting an environment variable `TESTTHAT_CPUS`, see: https://testthat.r-lib.org/articles/parallel.html.

To create a new test file, use:

```
use_test("my_function")
```

This creates `tests/testthat/test-my_function.R`, a template ready for adding tests.

18.1.2. Writing test blocks

Tests are written inside `test_that()` blocks:

```
test_that("my_function adds numbers correctly", {
  expect_equal(my_function(1, 2), 3)
})
```

A good test block:

- has a clear, descriptive name

- focuses on one coherent behaviour
- keeps heavy computation or complex setup outside the block

18.1.3. Running tests

During development, run your tests using:

```
library(devtools)
test()
```

Note all tests also run automatically during `R CMD check` (`check()`).

Tests will be run against the current loaded functions, so make sure that you run `load_all()` to load the current/latest version of your package.

18.2. Expectations

Expectations are the core of every test. They express the conditions that must be true for a function to behave correctly. If an expectation fails, testthat produces an informative message.

18.2.1. Common expectations

Frequently used expectations include:

- `expect_equal(x, y)`: checks that objects are equal (with tolerance)
- `expect_identical(x, y)`: strict equality including type and attributes
- `expect_true(x)` / `expect_false(x)`: checks logical conditions
- `expect_error(expr)` / `expect_warning(expr)` / `expect_message(expr)`: checks for specific conditions produced by code

- `expect_s3_class(x, "class")`: checks object class
- `expect_type(x, "double")`: checks base type

18.2.2. Why expectations matter

Expectations define the expected behaviour of your functions. They make assumptions explicit, illustrate expected output, and help maintainers understand the code. They also provide protection against regressions when internal implementations change.

A single `test_that()` block may contain several expectations, as long as they relate to the same behaviour.

18.3. Setup and Helper Functions

As your test suite grows, repeated patterns often emerge: constructing similar inputs, generating mock data, or creating temporary environments. To keep tests clear and maintainable, testthat provides a structured approach for managing shared setup code.

18.3.1. Helper files

Reusable functions should be placed in helper files. Any file named:

```
tests/testthat/helper-*.R
```

is executed before the tests run. These files are ideal for:

- mock datasets
- utility functions
- shared configuration

- frequently used objects

Example helper:

```
# tests/testthat/helper-data.R
make_mock_patient <- function(id = 1, gender = "M") {
  tibble::tibble(person_id = id, gender_concept_id = gender)
}
```

Tests can then use:

```
patient <- make_mock_patient()
```

18.3.2. Setup and teardown

For more complex requirements, testthat supports setup and teardown files:

- `setup-*.R` — executed before tests
- `teardown-*.R` — executed after tests

This pattern is useful to set up reusable mocks, database connections and so. We will see how to use those files in the Testing against multiple DBMS chapter to tests your package against multiple database management systems.

18.3.3. Avoid top-level code

You should avoid running code at the top level of test files. Instead, place code inside helpers or `test_that()` blocks. This makes tests more predictable and prevents shared state from leaking between tests.

18.4. What to Test

Good tests focus on what your package promises to do. Tests should be scoped to the behaviour that your package is responsible for—not on implementation details or on the behaviour of external code.

18.4.1. General guidelines

You should test:

- **Usual cases**: typical inputs your users will provide
- **Edge cases**: empty inputs, missing values, not usual combinations…
- **Error behaviour**: invalid inputs should produce informative errors
- **Output structure**: class, column names, attributes, lengths
- **Regression tests**: when a bug is fixed, add a test preventing its return, this is a very important step in test development so we do not encounter the same error once again.

> **Scope tests to your own code**
>
> Avoid testing behaviour of other packages, we want to check that our package not the other packages.

18.5. Documenting Tests

Tests should be readable and easy to understand. A well-written test suite functions as documentation for your package's expected behaviour.

18.5.1. Descriptive test names

The description passed to `test_that()` should be clear:

```
test_that("addAge correctly add age to a table", {
  ...
})
```

This helps identify failures and understand the intention behind the test.

18.5.2. Commenting test logic

Use comments to clarify:

- why specific inputs were chosen
- why an output is expected
- what bug a test prevents from reappearing (e.g. you can refer the issue where the problem was reported)
- key assumptions behind the test

Example:

```
# Missing values should be ignored; this mirrors the documented default.
expect_equal(mean_custom(c(1, 2, NA)), 1.5)
```

18.5.3. Keeping tests readable

To maintain readability:

- keep tests small and focused
- avoid deep nesting
- use helper functions for repeated patterns

- keep example datasets small and simple (we will see how to create mock OMOP CDM datasets in the Testing in the OMOP CDM chapter)

Readable tests make collaboration easier and reduce maintenance effort.

18.6. Test Coverage

Test coverage measures the proportion of your package's code that is executed during testing. It is expressed as the percentage of lines inside your `R/` folder that are run by the tests in `tests/testthat/`.

You can compute test coverage using:

```
test_coverage()
```

This generates an interactive report in the *Viewer* panel showing how many times each line of code was executed during testing, which lines are covered by tests, and, therefore, which lines remain untested.

High test coverage increases confidence that your functions behave as expected across typical and edge-case scenarios. While achieving **100% coverage** is ideal, it is not always practical; deprecated code paths or some edge-case checks may be difficult to test directly.

As a rule of thumb **aim for at least 90%-95% coverage**, in general, try to test all core logic, leaving only truly unavoidable lines uncovered.

Coverage is not a substitute for thoughtful testing, but it is a valuable tool for identifying weak spots in your test suite and ensuring your package remains reliable as it evolves. At the end having a 100% test does not ensures that your package does what you want, it just ensures that it does not break. So think carefully your tests and test the core functionality.

18.7. Testing on CRAN

When submitting a package to CRAN the package is checked (like `check()` function does), all tests, examples, vignettes, and documentation must finish within **10 minutes** of CPU time. Test can consume a substantial portion of this time limit, and it may exceed CRAN's limits.

To keep your CRAN checks within the allowed time while still maintaining a comprehensive local tests, you can selectively skip the most expensive tests on CRAN using `skip_on_cran()`:

test_that("Test mean_custom behaviour", { skip_on_cran() # Missing values should be ignored; this mirrors the documented default. expect_equal(mean_custom(c(1, 2, NA)), 1.5) }) ""

This approach allows you to run **full tests locally** and in continuous integration (e.g., GitHub Actions), but omit heavy or slow tests during CRAN checks, ensuring that only **core functionality** is tested on CRAN, while edge cases, performance tests, and large data scenarios are tested elsewhere.

> **Skip functions**
>
> The *testthat* package provides several functions for conditionally skipping tests.
> Common examples include:
>
> - `skip_on_cran()`: skip tests on CRAN only.
> - `skip_if()`: skip based on a custom condition (e.g. `skip_if(dbToTest == "Postgres")`).
> - `skip_if_not_installed()`: skip if a required package is missing (e.g. `skip_if_not_installed("dplyr")`).
>
> See the full list of skip helpers at: https://testthat.r-lib.org/reference/skip.html.

18.8. Further reading

- Wickham, H., & Bryan, J. *R Packages (2nd ed.): Testing basics (Chapter 13)*. Available online at: https://r-pkgs.org/testing-basics.html
- Wickham, H., & Bryan, J. *R Packages (2nd ed.): Designing your test suite (Chapter 14)*. Available online at: https://r-pkgs.org/testing-design.html
- Wickham, H., & Bryan, J. *R Packages (2nd ed.): Advanced testing techniques (Chapter 15)*. Available online at: https://r-pkgs.org/testing-advanced.html

19. Testing in the OMOP CDM

In this chapter we cover how to create synthetic data in the OMOP CDM format using the omock R package.

19.1. Using omock to create mock synthetic data

19.1.1. Mock data

You can create simple mock datasets using the different mock functions in omock, we will create the dataset in

- omock::mock functions

19.1.2. Mock datasets

The omock package also contains pre-build syntehtic datasets from Eunomia package. To use eunomia

- omock::mockCdmFromDataset

19.1.3. Minimum viable dataset

- omock::mockCdmFromTables

19.2. Inserting the mock

All those functions create local

19.3. Reusing mock data

19.4. Further reading

- omock paper
- omock package

20. Testing against multiple DBMS

In this chapter we c

Part V.

GitHub

GitHub is the central platform for developing, testing, and distributing packages in the Tidy R OMOP CDM ecosystem. Beyond version control, it provides the infrastructure for continuous integration: automatically running checks, building the documentation website, measuring test coverage, and testing against multiple database backends — all triggered on every push or pull request.

Setting this infrastructure up correctly from the beginning pays dividends throughout the life of a package. It ensures that regressions are caught immediately, that the documentation is always up to date, and that contributors can trust that their changes have been properly validated before merging.

This part of the book covers the GitHub setup used across the ecosystem. The chapters are organised as follows:

- Set up GitHub walks through the initial steps for connecting a package repository to GitHub: repository settings, branch protection, issue and pull request templates, and the conventions used across the ecosystem.
- Automatic R-CDM checks explains how to configure a GitHub Actions workflow that runs `R CMD check` on every push and pull request, including checks specific to OMOP CDM packages.
- Website deployment covers the GitHub Actions workflow for automatically building and deploying your pkgdown documentation website whenever changes are pushed to the main branch.
- Test coverage describes how to integrate Codecov with GitHub Actions to automatically measure and report test coverage on every pull request.
- Testing multiple DBMS explains how to configure GitHub Actions workflows that test your package against multiple database management systems — DuckDB, PostgreSQL, SQL Server — to ensure consistent behaviour across backends.

21. Set up GitHub

22. Automatic R-CDM checks

23. Website deployment

24. Test coverage

25. Testing multiple DBMS

Part VI.

Maintaining a package

Getting a package to a first release is a significant achievement, but it is only the beginning. Maintaining a package — keeping it working as its dependencies evolve, responding to issues, managing releases, and communicating changes clearly — is an ongoing commitment that requires its own discipline and practices.

The good news is that the infrastructure set up in the previous parts of this book (automated checks, test coverage, multi-DBMS testing) does a great deal of the maintenance work automatically. The chapters in this part cover the human side: the decisions and processes that cannot be automated.

This part currently contains one chapter:

- Submit to CRAN walks through the process of preparing and submitting a package to CRAN for the first time, and the ongoing practices for subsequent releases: deciding on version numbers, maintaining a `NEWS.md` changelog, checking reverse dependencies, and responding to CRAN policy changes.

Further chapters on longer-term maintenance practices — handling breaking changes, deprecation workflows, and managing a growing contributor community — will be added in future editions of this book.

26. Submit to CRAN

- What version to release (major, minor or patch)
- Forced releases by cran
- news
- reverse dependency